



RÍO NEGRO
UNIVERSIDAD NACIONAL

Proyecto Final Integrador
Ingeniería en Telecomunicaciones

Sistema para la detección y localización de objeto sonoro con detectores pasivos de sonido

Autor

Rodrigo Boyé

Director

Dr. Jorge Lugo

2026

Universidad Nacional de Río Negro
San Carlos de Bariloche, Río Negro, Argentina

Resumen

Frente al incremento en el uso de drones o Vehículos Aéreos No Tripulados (UAVs), cuya presencia puede generar problemas de seguridad, se requieren sistemas que sean capaces de detectar y localizar posibles amenazas.

La localización acústica es una solución eficiente y fácilmente desplegable que puede complementar a otros sistemas de localización (radar, lidar, radiofrecuencia, visual).

En este proyecto se diseñaron e implementaron dos sistemas de Localización de Fuente de Sonido (SSL): un sistema básico con un arreglo de dos micrófonos que utiliza el algoritmo GCC-PHAT para estimación en una dimensión (acimut o elevación) y un sistema con un arreglo cuadrado de cuatro micrófonos que aplica el algoritmo MUSIC para localizar en dos dimensiones (acimut y elevación). El sistema de cuatro micrófonos logró detectar la firma acústica de un dron y localizarlo de forma consistente en condiciones favorables (SNR mayor a 10 dB, distancia de cinco metros en un ambiente abierto), con rendimiento degradado a distancias mayores y ambientes ruidosos o reverberantes. El sistema de cuatro micrófonos también puede aplicar el algoritmo SRP-PHAT para la localización de eventos impulsivos, como podría ser la detonación de artillería, pero esta aplicación no fue medida.

Durante el desarrollo del trabajo se realizó un estudio de la disciplina, se revisaron sistemas similares, se simularon distintas configuraciones y algoritmos y se implementaron los diseños finales. Todas estas etapas, junto a los resultados y conclusiones obtenidos, se incluyen en el presente documento.

Palabras clave: SSL, MUSIC, GCC-PHAT, SRP-PHAT, DOA, UAV, procesamiento de array.

Agradecimientos

A Invap, por proporcionar el tema y los medios para realizar el trabajo.

Al director de este proyecto, por su guía, su ayuda y amabilidad.

A la Universidad, sus docentes, no docentes y compañeros, por formarme como ingeniero y enseñarme tantas otras cosas.

A mis amigos, en Allen y en Bariloche, por tantas risas, por su amistad, y compañía en todos estos años.

A mi familia, especialmente a mis padres y abuelos, que me dieron todo lo que una persona puede necesitar y mucho más.

A todos, por su apoyo, su paciencia y su confianza, gracias de corazón.

Índice de Contenidos

Resumen	I
Agradecimientos	II
Índice de Figuras	VI
Índice de Tablas	IX
Acrónimos	X
Glosario	XIII
1. Introducción	1
1.1. Motivación	1
1.2. Objetivos y alcance	2
1.3. Organización	4
1.4. Notación y convenciones	4
2. Marco teórico y estado del arte	6
2.1. Sonido	6
2.2. Arreglo de sensores	8
2.3. Detección y localización	11
2.3.1. Clasificación	11
2.3.2. Parámetros	12
2.4. Estimación de Dirección de Arribo	14
2.4.1. Estudio general	14
2.4.2. Algoritmo GCC-PHAT	17
2.4.3. Algoritmo SRP-PHAT	19

2.4.4. Algoritmo MUSIC	20
2.5. Análisis de sistemas similares para el diseño del sistema	28
2.6. Resumen del capítulo	29
3. Simulaciones	31
4. Diseño e implementación	37
4.1. Arreglo de dos micrófonos con GCC-PHAT	37
4.1.1. Cadena de procesamiento	38
4.1.2. Decisiones de diseño, limitaciones y posibles mejoras . . .	40
4.2. Arreglo de cuatro micrófonos con MUSIC	40
4.2.1. Cadena de procesamiento	41
4.2.2. Decisiones de diseño, limitaciones y posibles mejoras . . .	44
4.2.3. Comparación MUSIC y SRP-PHAT	47
4.3. Resumen y comparación	48
5. Pruebas y resultados	50
5.1. Sistema de dos micrófonos	50
5.2. Sistema de cuatro micrófonos	51
6. Conclusiones y posibles líneas futuras	55
6.1. Conclusiones generales	55
6.2. Posibles líneas futuras	56
A. Descomposición en autovalores	59
B. Dispositivos y herramientas utilizadas	61
B.1. Dispositivos utilizados	61
B.1.1. Micrófonos INMP441	61
B.1.2. ESP32	64
B.1.3. Raspberry Pi 3A+	64
B.1.4. Dron DJI Mini 4 Pro	65
B.2. Herramientas utilizadas	66
B.2.1. Matlab	66
B.2.2. ArduinoIDE	66

B.2.3. Visual Studio Code (VS Code)	67
B.2.4. EasyEDA	67
B.2.5. Fresadora CNC	67
C. Esquemáticos y PCBs	68
C.1. Esquemáticos	68
C.2. PCBs	70
D. Programas utilizados en el diseño final	72
D.1. ESP32_audio_frontend	73
D.2. config	74
D.3. audio_input	75
D.4. detector	75
D.5. spectral_gate	77
D.6. doa_engine	78
D.7. main	80
Bibliografía	83

Índice de Figuras

2.1. Múltiples direcciones de arribo debido a multipathing.	8
2.2. Ejemplos de configuración geométrica lineal (a), planar (b) y volumétrica (c).	9
2.3. Arreglos utilizados para el sistema 1 (a) y 2 (b).	9
2.4. Coordenadas esféricas (acimut φ , elevación θ y distancia r). . . .	12
2.5. Trilateración (a), hipérbolas (b) y triangulación (c) [Zaidi, 2010]. .	14
2.6. Señal incidiendo en un ULA de M elementos [Chen, 2010].	15
2.7. Ejemplos de pseudoespectros MUSIC para una y dos dimensiones.	22
2.8. Diagrama en bloque del algoritmo MUSIC implementado.	24
2.9. Interpolación parabólica.	26
2.10. Proyección del espacio- u en el espacio angular. Rango y cantidad de puntos utilizados en el sistema de cuatro micrófonos	28
3.1. Arreglo ULA de dos elementos para simulación.	31
3.2. Pseudoespectros para distintos algoritmos en ULA de dos elementos.	32
3.3. Resolución de MVDR y MUSIC para dos señales cercanas en ULA de 20 elementos.	32
3.4. Efectos del aliasing espacial.	33
3.5. Arreglo URA de cuatro elementos para simulación.	34
3.6. Resolución de MUSIC en <i>broadside</i> y <i>endfire</i> para URA de 4 elementos.	34
3.7. Arreglo tetraédrico de cuatro elementos para simulación.	35
3.8. Resolución de MUSIC en distintos ángulos para arreglo tetraédrico de 4 elementos.	36

4.1. La implementación final. De izquierda a derecha: el arreglo de 4 micrófonos, el arreglo de 2 micrófonos, la Raspberry Pi y los servomotores.	37
4.2. Vista frontal y trasera del sistema de dos micrófonos.	38
4.3. Cadena de procesamiento del sistema de 2 micrófonos.	38
4.4. Vista frontal y trasera del segundo diseño implementado.	41
4.5. Cadena de procesamiento del sistema de 4 micrófonos.	41
4.6. Estructura del paquete serial por hop (de 256 muestras, modificable).	41
4.7. Estimación de BPF mediante Espectro de Productos Armónicos (fuente: musicweb.ucsd).	43
4.8. Espectro de frecuencia de un dron real. Se aprecia claramente el BPF en 250 Hz y los armónicos de mayor magnitud dentro del rango de frecuencias de MUSIC (200–2400 Hz).	44
4.9. Espectro de la reproducción de la grabación de un dron desde el parlante de un celular; se pierden los bajos y el comportamiento armónico, no activa el detector espectral.	44
4.10. Traslado de rango con inclinación del arreglo.	45
5.1. Sistema de dos micrófonos en muestra de Invap.	51
5.2. Prueba del sistema de cuatro micrófonos con un dron en campo abierto.	53
5.3. Simulación: tasa de error mayor a 10° en MUSIC según SNR.	53
5.4. Simulación: raíz de error cuadrático medio (RMSE) en MUSIC según SNR y ángulo de arribo.	54
B.1. Módulo breakout del INMP441.	62
B.2. Diagrama en bloques del INMP441.	62
B.3. Output I2S estéreo (arriba) y mono (abajo).	63
B.4. Configuración estéreo de dos micrófonos INMP441.	63
B.5. Placa ESP32.	64
B.6. Placa Raspberry Pi 3A+.	65
B.7. Dron DJI Mini 4 Pro.	66
C.1. Plano esquemático del sistema de dos micrófonos.	68
C.2. Plano esquemático del sistema de cuatro micrófonos.	69

C.3. PCB del sistema de dos micrófonos.	70
C.4. PCB del sistema de cuatro micrófonos.	71
D.1. Captura de pantalla de la interfaz del sistema por terminal	82

Índice de Tablas

2.1. Tipos de algoritmos clásicos de estimación de DOA.	17
4.1. Comparación entre sistemas.	49
5.1. Estimación con MUSIC en sistema de cuatro micrófonos, ambiente cerrado, distancia corta.	52

Acrónimos

AWGN

Ruido Blanco Aditivo Gaussiano (*Additive White Gaussian Noise*). Modelo de ruido aleatorio que no varía con la frecuencia.

BPF Frecuencia de Paso de Pala (*Blade Passing Frequency*). Tono característico de un rotor, igual a número de palas $\times$ RPM/60.

DOA Dirección de Arribo (*Direction of Arrival*). Parámetro central del trabajo: la dirección de la que proviene la señal.

ESPRIT

Estimation of Signal Parameters via Rotational Invariance Techniques. Método de subespacio que explota la invarianza estructural del arreglo.

EVD *Eigenvalue Decomposition* (Descomposición en Autovalores). Operación algebraica central en el algoritmo MUSIC, desarrollada en el Apéndice A.

FDOA Diferencia de Frecuencia de Arribo (*Frequency Difference of Arrival*).

FFT Transformada Rápida de Fourier (*Fast Fourier Transform*).

GCC-PHAT

Correlación Cruzada Generalizada con Transformada de Fase (*Generalized Cross-Correlation with Phase Transform*). Algoritmo del sistema 1.

GPIO Entrada/Salida de Propósito General (*General Purpose Input/Output*). Pines de la Raspberry Pi.

HPS Espectro de Productos Armónicos (*Harmonic Product Spectrum*). Usado por el detector espectral para estimar la BPF.

I2S *Inter-IC Sound*. Bus serie para audio digital entre circuitos integrados.

IDE Entorno de Desarrollo Integrado (*Integrated Development Environment*).

IR	Infrarrojo.
LOS	Línea de Visión (<i>Line of Sight</i>).
MEMS	Sistemas Microelectromecánicos (<i>Micro-Electro-Mechanical Systems</i>).
ML	Máxima Verosimilitud (<i>Maximum Likelihood</i>) o también <i>Machine Learning</i> .
MUSIC	Clasificación de Señales Múltiples (<i>MUltiple SIgnal Classification</i>). Algoritmo principal del sistema 2.
MVDR	Respuesta de Mínima Varianza sin Distorsión (<i>Minimum Variance Distortionless Response</i>).
PCB	Placa de Circuito Impreso (<i>Printed Circuit Board</i>).
PCM	Modulación por Impulsos Codificados (<i>Pulse Code Modulation</i>). Formato de datos de la salida I2S.
PFI	Proyecto Final Integrador.
PPS	Práctica Profesional Supervisada.
PWM	Modulación por Ancho de Pulso (<i>Pulse Width Modulation</i>). Control de los servomotores.
RF	Radiofrecuencia.
RMSE	Raíz del Error Medio Cuadrático (<i>Root Mean Square Error</i>). Se eleva el error al cuadrado para que no se cancelen positivos y negativos, y para penalizar los errores grandes, se promedia y luego se toma la raíz para volver a las unidades originales.
ROI	Región de Interés. La banda de frecuencia de trabajo dentro de todo el espectro.
RPM	Revoluciones por minuto.
SCK / SD / WS / CLK	Líneas del bus I2S: reloj serial de bit (SCK/CLK), datos de salida (SD) y selección de palabra (WS).

SELD Detección y Localización de Evento Sonoro (*Sound Event Localization and Detection*).

SNR Relación Señal-Ruido (*Signal-to-Noise Ratio*).

SRP-PHAT

Potencia de Respuesta Dirigida con Transformada de Fase (*Steered Response Power Phase Transform*). Algoritmo secundario del sistema 2.

SSH *Secure Shell*. Protocolo de acceso remoto usado para editar sobre la Raspberry Pi.

SSL Localización de Fuente de Sonido (*Sound Source Localization*).

TDOA

Diferencia de Tiempo de Arribo (*Time Difference of Arrival*).

TOA Tiempo de Arribo (*Time of Arrival*).

UAV Vehículo Aéreo No Tripulado (*Unmanned Aerial Vehicle*).

ULA Arreglo Lineal Uniforme (*Uniform Linear Array*).

URA Arreglo Rectangular Uniforme (*Uniform Rectangular Array*).

USB *Universal Serial Bus*.

WSN Red de Sensores Inalámbricos (*Wireless Sensor Network*).

Glosario

Acimut/azimut

Ángulo horizontal de la dirección de arriba dentro del sistema de coordenadas esféricas.

Aliasing espacial

Ambigüedad angular que aparece cuando la separación entre elementos supera media longitud de onda, análogo espacial del aliasing temporal.

Broadside

Dirección perpendicular al arreglo (el frente), donde la sensibilidad angular es máxima.

Campo cercano / campo lejano

Distancia de propagación desde la fuente: en campo lejano los frentes de onda se asumen planos y vale la estimación.

Elevación

Ángulo vertical de la dirección de arriba en coordenadas esféricas.

Endfire

Dirección paralela al arreglo (los costados), donde la sensibilidad angular es mínima.

Espacio-u

Grilla de escaneo donde la coordenada es el seno del ángulo; concentra puntos en torno al eje normal del arreglo (*broadside*).

Firma acústica

Conjunto de rasgos espectrales que identifican una fuente. Por ejemplo, el “peine” armónico de un dron.

Frente de onda

Superficie de fase constante de la onda, se asume plano en campo lejano.

Hop Paquete de muestras nuevas que entran en cada iteración, por default de 256 muestras por canal.

Matriz de covarianza espacial

Matriz que cuantifica la correlación entre las señales de los elementos del arreglo.

Multipathing / reverberación

Llegada de la señal por múltiples caminos tras reflejarse; introduce interferencias y errores de DOA.

Pseudoespectro

Función cuyos picos indican las direcciones estimadas de las fuentes (salida de MUSIC/SRP-PHAT).

Resolución (angular)

La menor separación angular entre dos fuentes que el sistema puede distinguir.

Snapshot

Observación instantánea del vector de salida del arreglo, vector complejo con un elemento para cada sensor.

Subespacio de señal / de ruido

Descomposición ortogonal de la matriz de covarianza; MUSIC explota que la dirección verdadera es ortogonal al subespacio de ruido.

Vector de steering (o de dirección)

Vector que describe cómo responde cada elemento del arreglo a un ángulo de arribo dado.

Capítulo 1

Introducción

1.1. Motivación

El Proyecto Final Integrador (PFI) es el requerimiento final para acreditar el título de ingeniero en telecomunicaciones. En este caso es una continuación de la Práctica Profesional Supervisada (PPS), llevada a cabo entre octubre de 2024 y mayo de 2025 en la empresa Invap, que consistió en el estudio, investigación y una presentación sobre el tema. Algunos de los resultados de la PPS están presentes en el capítulo 2, marco teórico y estado del arte.

En las telecomunicaciones, los sistemas de detección y localización son aquellos que se encargan de descubrir la existencia de una señal particular y estimar o calcular la ubicación en el espacio desde donde proviene. Pueden referir a señales electromagnéticas o acústicas, a cualquier escenario y a cualquier técnica o método de localización, por lo que es un campo de estudio interdisciplinario muy amplio, con gran cantidad de aplicaciones diferentes. Algunos sistemas que realizan localización son los radares y sonares, los GPS, el beamforming en antenas inteligentes, el monitoreo de fauna y la localización de personas, entre otros (Li et al., 2016; Jekateryńczuk & Piotrowski, 2023).

El sistema desarrollado en este trabajo está motivado por el requerimiento de Invap de detectar y localizar señales acústicas, principalmente proveniente de drones o UAVs, pero también aplicable a otros eventos como detonaciones.

La presencia de un dron puede detectarse visualmente, con radar, por infrarrojo (IR), por radiofrecuencia (RF) o acústicamente; las ventajas de utilizar el medio acústico son (Case et al., 2008; Jekateryńczuk & Piotrowski, 2023; Ramamonjy et al., 2018; Riabko et al., 2024):

- **Independencia del tamaño del objetivo:** los drones pequeños presentan dificultades para la detección visual o por radar pero, si bien existen modelos relativamente silenciosos, la emisión acústica de sus motores y hélices no

puede suprimirse por completo, por lo que sigue siendo posible la detección acústica.

- **Sistemas pasivos y furtivos:** se limitan a escuchar, no transmiten señales hacia el objetivo, esto permite operar de forma remota sin revelar su presencia o posición.
- **Operatividad en cualquier condición:** a diferencia de otros sistemas que pueden fallar por obstrucción física, oscuridad o condiciones climáticas, en los sistemas acústicos las ondas se propagan alrededor de los obstáculos y en la oscuridad; sin embargo se ve afectado por los efectos auditivos de fenómenos climáticos como el viento, la lluvia, truenos.
- **Simplicidad y bajo costo:** se pueden construir sistemas de forma relativamente simple y con hardware de bajo costo, como se evidencia en este trabajo, comparado a algunos de los demás sistemas.
- **Capacidad de clasificación:** las señales acústicas de los UAVs contienen información que permite identificar características particulares del objetivo como el modelo o el tamaño. Se está avanzando mucho en este campo mediante redes neuronales y algoritmos de aprendizaje automático.
- **Inmunidad a (algunas) interferencias y jamming:** cuando un objetivo malicioso no quiere ser detectado o localizado puede utilizar distintas técnicas para “engañar” o “ahogar” a sistemas de radar y radiofrecuencia (reenviar la señal de radar modificada, sobrecargar el espacio de ondas electromagnéticas, entre otras). Si bien teóricamente se pueden aplicar técnicas similares sobre los sistemas acústicos, es más difícil de ejecutar y ocultar.

A pesar de todas estas ventajas, los sistemas acústicos distan de ser perfectos o invulnerables. Es por eso que se recomienda su utilización como complemento dentro de un sistema de sistemas de detección, siendo una excelente primera línea de detección temprana que puede alertar y dirigir a sistemas con buena precisión pero con campo de visión restringido hacia el objetivo.

1.2. Objetivos y alcance

El objetivo principal de este PFI es el de diseñar e implementar un sistema de estimación de Dirección de Arribo (DOA) de una señal acústica en tiempo real utilizando un arreglo de micrófonos y procesamiento de señales sobre hardware de bajo costo. Los tipos de fuente considerados son en primer lugar un dron o

UAV, y en segundo lugar detonaciones de artillería. De este objetivo general se desprenden los objetivos específicos de: capturar de forma sincronizada cuatro canales de audio, transmitir el audio hasta la unidad de procesamiento, detectar la presencia de un “evento sonoro” relevante en el ruido de fondo e implementar un algoritmo de estimación de DOA sobre la señal del evento. Adicionalmente, se plantearon los objetivos complementarios de controlar un par de servomotores para que apunten a la dirección estimada y el de generar un registro de eventos.

La realización de este proyecto incluyó el estudio teórico de los principios físicos y matemáticos implicados, de los algoritmos, de las características y el uso de los componentes, de las configuraciones de los arreglos y las técnicas de localización; también requirió realizar diferentes simulaciones, pruebas, mediciones y correcciones para llegar al diseño final. Este documento es el registro del trabajo realizado y de los resultados obtenidos.

En cuanto al alcance, se llegó a un sistema que puede estimar la dirección de arriba en dos dimensiones (acimut y elevación). Las características del arreglo permiten estimar de una a tres fuentes, pero para este trabajo solo vamos a trabajar con una única fuente ya que permite lograr resultados más concretos, el sistema no fue caracterizado para más de una fuente.

El sistema escanea una grilla de, por default, 31 puntos de acimut y 13 de elevación; 403 puntos uniformes en el seno de cada ángulo y por lo tanto no uniformes en grados (espacio-u, desarrollado en la sección 2.4). Los pasos son de 5° en acimut y 6° en elevación en el eje del arreglo, aumentando progresivamente hacia los bordes. Sobre esos pasos se alcanza resolución sub-grilla mediante interpolación parabólica de tres puntos. El rango de escaneo en acimut es de $\pm 75^\circ$ y el de elevación abarca $\pm 35^\circ$ respecto al eje del arreglo, pudiendo inclinarse para trasladar el rango a, por ejemplo, $[0^\circ, 70^\circ]$. Los límites de los rangos y la cantidad de puntos de la grilla responden a limitaciones físicas del arreglo y a la capacidad de cómputo del sistema. Estas limitaciones se explican y desarrollan en el informe.

No se pudo realizar una medición cuantitativa precisa del rendimiento del sistema debido a la dificultad de apuntar una fuente desde una dirección exacta para comparar con la estimación y la variación de rendimiento dependiendo del ambiente, pero simulando señales, corriéndolas por el *pipeline* se observó que para SNR de 10 dB o mayor el RMSE es menor a 5° . La precisión se degrada en condiciones ruidosas, ambientes reverberantes y en los límites del rango.

1.3. Organización

El texto está organizado de la siguiente forma: el primer capítulo es la introducción donde se presentan los objetivos y el tema; el segundo capítulo, un marco teórico y estado del arte donde se desarrollan los conceptos que conciernen al trabajo y se sacan conclusiones del análisis de diseños similares; el tercer capítulo registra las simulaciones realizadas para familiarizarse con los sistemas; el cuarto capítulo describe cómo se diseñó el sistema, el por qué de las decisiones de arquitectura, configuración, algoritmo y componentes; el quinto capítulo presenta los resultados de las pruebas de los dos prototipos; finalmente, en el sexto capítulo se encuentran las conclusiones extraídas del trabajo y sus resultados, junto con propuestas para mejorar o continuar el proyecto. En los apéndices se incluyen: el proceso matemático sobre el que se basa el algoritmo MUSIC, una lista y descripción de los dispositivos y herramientas utilizados en el desarrollo del trabajo, los planos esquemáticos y las placas PCB diseñadas, y una descripción de los *scripts* utilizados para el funcionamiento del sistema.

1.4. Notación y convenciones

A lo largo del documento se adopta la siguiente convención para la notación matemática, con el fin de distinguir escalares, vectores y matrices:

- **Escalares:** letras en itálica, por ejemplo f (frecuencia), τ (retardo) o θ (ángulo).
- **Vectores:** letras minúsculas en negrita, por ejemplo $\mathbf{x}(t)$ para el vector de señales recibidas o $\mathbf{a}(\mu)$ para el vector de dirección (*steering*).
- **Matrices:** letras mayúsculas en negrita, por ejemplo $\mathbf{A}$ (matriz de dirección), $\mathbf{R}_{xx}$ (matriz de covarianza espacial) o $\mathbf{V}_n$ (subespacio de ruido).
- **Ángulos:** se emplea φ para el acimut y θ para la elevación.
- **Dimensiones e índices:**
 - M : cantidad de micrófonos en un arreglo.
 - I : cantidad de fuentes o señales incidentes.
 - N : cantidad de *snapshots* promediados en la covarianza.
 - n : índice de muestras dentro de un vector de señal discreta.
 - k : índice de bins de frecuencia de la FFT.

Se emplean además los operadores $(\cdot)^T$ para la transpuesta, $(\cdot)^H$ para la transpuesta conjugada (operador hermítico), $\mathbb{E}\{\cdot\}$ para la esperanza estadística y $|\cdot|$ para el módulo. Los símbolos y unidades específicas se introducen a medida que aparecen en el texto.

Capítulo 2

Marco teórico y estado del arte

Se explicarán los conceptos teóricos detrás de los métodos de localización utilizados, desde lo más general o fundamental a lo más específico, y luego se presentarán brevemente unas conclusiones extraídas de la revisión de sistemas similares.

2.1. Sonido

Trabajaremos con señales acústicas, por lo que es necesario definir y entender qué son y cómo las modelamos.

El sonido es un fenómeno físico, vibraciones de presión transmitidas a través de medios elásticos como el aire (no puede propagarse por el vacío). Se puede representar matemáticamente como una onda sinusoidal

$$x(t) = A \cdot \text{sen}(\omega t + \varphi) \quad (2.1)$$

donde A es la amplitud, ω es la frecuencia angular (igual a $2\pi \cdot f$), t es el tiempo y φ es la fase.

La ecuación (2.1) representa a un tono de una sola frecuencia, pero en la práctica se encuentran señales compuestas por múltiples frecuencias, por lo que una expresión más realista es

$$x(t) = \sum_{i=1}^N A_i \cdot \text{sen}(\omega_i t + \varphi_i) \quad (2.2)$$

donde hay N tonos presentes al mismo tiempo, cada uno con su respectiva amplitud A , frecuencia f (número de oscilaciones por segundo, en Hertz [Hz], igual a $\omega/2\pi$) y fase φ (diferencia en grados entre el estado del ciclo de la señal con un punto de referencia). Se puede descomponer una señal periódica compuesta por sinusoides de muchas frecuencias utilizando la Transformada de Fourier.

Las señales electromagnéticas pueden ser representadas de la misma forma que las señales acústicas y es por eso que los sistemas de localización para ambos tipos de señal son conceptualmente casi idénticos. Se diferencian, sin embargo, en la forma de percibir las señales, las frecuencias típicas, el ancho de banda, la velocidad de propagación y los medios por los que se pueden propagar (las ondas electromagnéticas sí pueden viajar por el vacío); esto implica diferencias en la implementación de cada tipo de sistema.

Las frecuencias audibles por el ser humano abarcan aproximadamente la banda entre 20 Hz a 20 kHz, pero en este proyecto trabajaremos dentro de la banda de 200 Hz a 2,4 kHz ya que es donde se concentra la mayor parte de la energía de la señal acústica de los UAVs (Zhu et al., 2021) y por limitaciones físicas del arreglo que se desarrollan más adelante. Estas frecuencias están dentro de la banda de trabajo los micrófonos utilizados (INMP441, *Apéndice B.1.1*), por lo que pueden captarse normalmente. Una señal dentro de este rango de frecuencias se considera de Banda Ancha o *wideband*: su ancho de banda es grande comparado con su frecuencia central. Que sea de banda ancha implica que sus distintas frecuencias tendrán diferentes desfases, lo que se debe tener en cuenta al elegir el algoritmo de localización a utilizar.

La velocidad de propagación del sonido en el aire se aproxima con 343 m/s, o está dada con mayor precisión por la ecuación (2.3). Conocer la velocidad de propagación v y la frecuencia f nos permite conocer la longitud de onda λ a través de la relación (2.4). La longitud de onda es determinante para el diseño del sistema ya que definirá las dimensiones físicas del arreglo.

$$v = 331 + 0,6 \cdot T[^\circ\text{C}] \quad [\text{m/s}] \quad (2.3)$$

$$\lambda = \frac{v}{f} \quad [\text{m}] \quad (2.4)$$

Se debe tener en cuenta que existen fenómenos que pueden afectar la propagación de la onda entre el emisor y el receptor, por lo que la dirección de arribo de la señal no coincide necesariamente con la ubicación de la fuente: la señal puede ser reflejada o refractada en superficies y bordes, modificando su trayectoria. El multipathing, propagación multitrayecto o reverberación es el fenómeno que ocurre cuando la onda llega a los sensores desde distintos caminos, puede causar interferencias y errores en la estimación de DOA (Fig. 2.1).

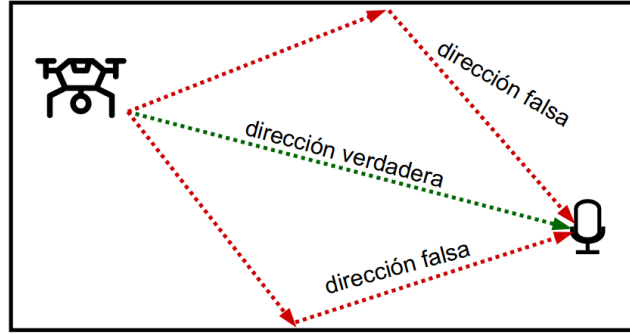


Figura 2.1: Múltiples direcciones de arribo debido a multipathing.

2.2. Arreglo de sensores

Se le llama arreglo o *array* a un grupo de sensores (o antenas) que trabajan en conjunto dispuestos en algún patrón geométrico, posiciones conocidas del espacio. Medir simultáneamente en posiciones diferentes implica que las mediciones sean ligeramente distintas, y esa diferencia, que depende de la dirección de la onda y de la geometría del arreglo, es la información que nos permite estimar la DOA. El concepto de arreglos receptores y/o emisores tiene muchas otras aplicaciones entre las que podemos destacar el beamforming, la discriminación de fuentes o mejoras de SNR.

Hay 4 asuntos de interés principales para el procesamiento de arrays (Van Trees, 2002):

A. Configuración del arreglo: consiste tanto en el patrón de radiación de cada elemento como en la configuración geométrica del arreglo. Considerando un patrón omnidireccional y enfocándonos en la geometría, podemos definir 3 categorías: lineal, planar y volumétrica (Fig. 2.2).

Las dimensiones en las que podemos estimar la DOA dependen directamente de las dimensiones sobre las que se extienda el arreglo: un arreglo lineal distingue una dimensión; un arreglo planar, dos; y un arreglo volumétrico puede además diferenciar atrás/adelante (utilizando coordenadas esféricas, explicado en la siguiente sección). La cantidad de elementos impacta en la precisión del sistema y la cantidad de fuentes que puede distinguir. Para este trabajo se construyó un arreglo lineal de dos elementos y un arreglo cuadrado/circular de cuatro elementos (para cuatro elementos, cuadrado y circular son equivalentes, Fig. 2.3) por lo que podrán distinguir una y dos dimensiones respectivamente, pero ninguno puede resolver la ambigüedad atrás/adelante.

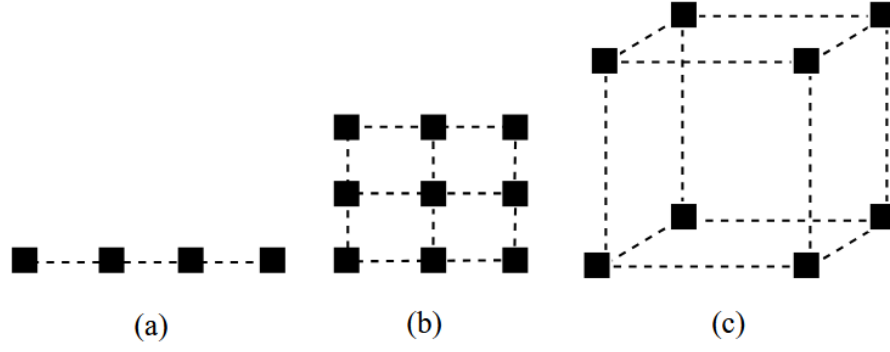


Figura 2.2: Ejemplos de configuración geométrica lineal (a), planar (b) y volumétrica (c).

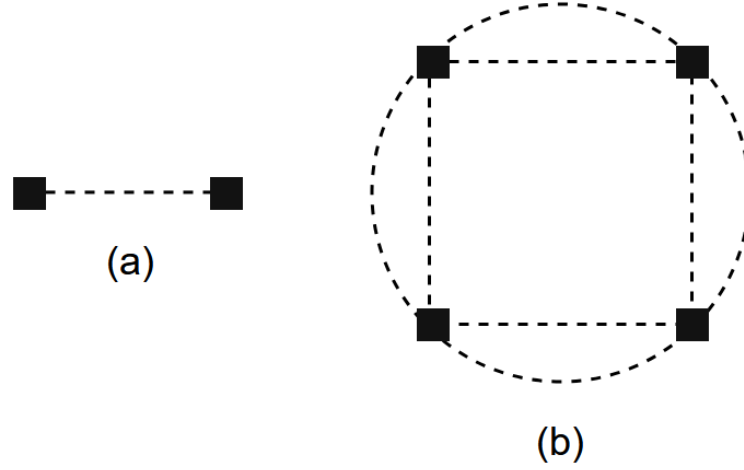


Figura 2.3: Arreglos utilizados para el sistema 1 (a) y 2 (b).

La apertura del arreglo se define como la distancia entre los elementos más alejados, es el tamaño del array, y afectará, junto con la cantidad de elementos, a la resolución: mientras mayor sea la distancia d entre micrófonos, mayor diferencia habrá entre las señales medidas para dos ángulos similares, por lo que podremos distinguirlos con mayor precisión.

Aumentar la apertura mejora la resolución angular pero disminuye la frecuencia máxima que podemos estimar sin ambigüedad. De forma similar a como se produce aliasing temporal, el fenómeno en el cual dos señales de frecuencias distintas no se pueden distinguir cuando la frecuencia de muestreo no supera el doble de la frecuencia más alta (Teorema de Nyquist), se puede producir un aliasing espacial cuando muestreamos un frente de onda a una distancia demasiado grande: si la separación entre elementos es mayor a la mitad de la longitud de onda de la señal (2.5), el desfase

observado puede ser superior a 180° y generar ambigüedad ya que hay más de una dirección de arribo compatible con ese valor (“¿Es una señal adelantada 190° o una atrasada 170° ?”).

$$d \leq \frac{\lambda_{f_{\text{máx}}}}{2} \quad \text{ó} \quad f \leq \frac{v}{2d} \quad (2.5)$$

Entonces, el compromiso entre resolución angular y límite de frecuencia será lo que regirá las dimensiones físicas del arreglo. En nuestro caso, el arreglo cuadrado se diseñó con 5 cm de lado, por lo que la distancia más larga, la diagonal, es de $5\sqrt{2} \approx 7,07$ cm y la frecuencia máxima sin ambigüedad es de 2425 Hz. Como se mencionó, consideramos que la mayoría de la información de las señales que utilizaremos está entre los 200 Hz y 2,4 kHz, por lo que estas dimensiones maximizan la resolución angular para la banda utilizada.

Se le llama broadside a la dirección perpendicular al arreglo (el frente, donde los ángulos de arribo son cercanos a 0° respecto a la normal) y endfire a la dirección paralela al mismo (los costados, donde los ángulos se aproximan a $\pm 90^\circ$). La sensibilidad angular es máxima en broadside ya que un pequeño cambio de ángulo produce un cambio grande en las diferencias entre señales recibidas, y mínima en endfire donde sucede lo contrario. Por esto los sistemas trabajan mejor cerca del frente y desestiman o tratan con mayor cuidado los extremos.

B. Características temporales y espaciales de la señal: debemos saber si estamos trabajando con señales conocidas, señales con parámetros desconocidos, señales con estructura conocida (por ejemplo, cierta modulación), o señales aleatorias. En este caso la señal es conocida (una onda de sonido dentro de banda de frecuencias determinadas y con comportamiento armónico) con parámetros desconocidos.

C. Características temporales y espaciales de la interferencia: si es una señal “conocida” pero con parámetros desconocidos, o si es ruido aleatorio. En principio modelamos la interferencia esperada como ruido blanco aditivo gaussiano (AWGN).

D. El objetivo del problema de procesamiento: para qué utilizamos el arreglo, en este caso para detectar una señal en presencia de ruido y estimar la dirección de arribo.

No se debe confundir los arreglos de sensores con las Redes de Sensores Inalámbricos (WSN) donde los sensores están distribuidos geográficamente y colaboran

de forma individual al procesamiento; los arreglos están ubicados relativamente cerca y el procesamiento es centralizado (las señales se combinan). Existen, y son comunes en los sistemas de localización, las redes de arreglos donde se distribuye una red de varios nodos, cada uno de ellos un arreglo.

2.3. Detección y localización

Los términos detección y localización pueden tomar muchos significados dependiendo de la disciplina en la que nos encontremos. En nuestro caso, detección se refiere al descubrimiento de la existencia de una señal dentro de un ambiente que puede ser ruidoso, mientras que localización significa identificar el punto o área específica en el espacio físico desde donde proviene esa señal. La importancia de realizar la detección antes de la localización está en evitar el gasto de energía y cómputo de procesar ruido de fondo, sin ninguna fuente de interés.

Los sistemas de Detección y Localización de Evento Sonoro (SELD) o Localización de Fuente de Sonido (SSL) (término muy similar que no considera o da por sentada la detección) tienen aplicaciones en muchas áreas civiles, militares y científicas. Se pueden utilizar en conjunto con otros sistemas de localización como radares o sistemas de localización por video; la ventaja que ofrecen es que pueden tener una buena precisión y robustez a un costo relativamente bajo (Jekaterýńczuk & Piotrowski, 2023).

2.3.1. Clasificación

Podemos clasificar los sistemas SELD según varios criterios, entre ellos:

- **Según el número de micrófonos:** monoaural, binaural, trinaural, etc.
- **Según capacidades de localización espacial:** estimar una, dos o tres dimensiones de la ubicación de la fuente. Se suelen utilizar coordenadas polares esféricas, por lo que estas dimensiones son dos ángulos que determinan la dirección (acimut y elevación) y una magnitud que completa la ubicación (la distancia, Fig. 2.4).
- **Según el número de fuentes que puede detectar:** el límite teórico para la cantidad I de fuentes que un arreglo de M elementos puede distinguir con algoritmos clásicos está dado por (2.6) (Friedlander, 2009; Chen et al., 2010)

$$I \leq M - 1 \tag{2.6}$$

sin embargo, este límite puede verse disminuido cuando las señales están altamente correlacionadas entre sí (reflexiones, fuentes similares) o son muy débiles.

- **Según la geometría del arreglo:** lineal, circular, hexagonal, piramidal, ad-hoc, etc.
- **Según el tipo de método utilizado:** los sistemas “clásicos” se basan en modelos matemáticos donde se siguen algoritmos para el procesamiento, pero en los últimos años han tomado impulso los métodos de Inteligencia Artificial (IA), los cuales son entrenados con datasets y son capaces de reconocer patrones, características y parámetros en nuevas señales. Este trabajo se concentra en los métodos clásicos en general y en los algoritmos GCC-PHAT y MUSIC en particular; queda abierta la posibilidad de reemplazar o complementar el método con IA en el futuro.

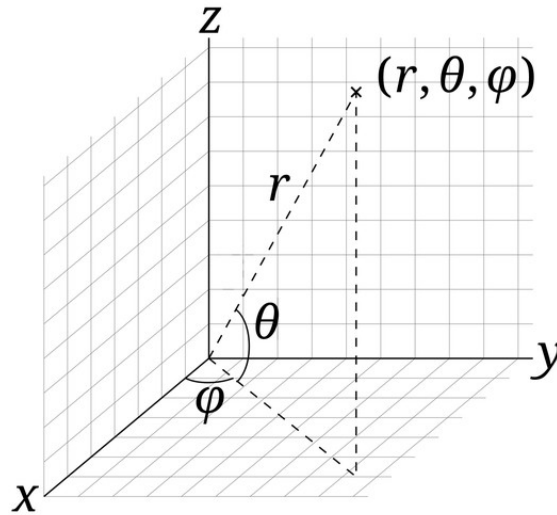


Figura 2.4: Coordenadas esféricas (acimut φ , elevación θ y distancia r).

2.3.2. Parámetros

Se utilizan distintos parámetros extraídos de la señal con el arreglo para realizar la localización. Dependiendo de cada situación, algunos serán más prácticos y otros imposibles de obtener, por lo que debemos elegir con precaución cuál o cuáles usar. Los principales parámetros son:

- **Dirección o Ángulo de Arribo (DOA o AOA):** el ángulo con el que las ondas impactan sobre los sensores en acimut y/o elevación. Para obtenerlo no se requiere ninguna sincronización con la fuente por lo que se utiliza, como es nuestro caso, cuando la fuente es desconocida. Es sensible

al multipathing, así que es preferible en ambientes abiertos y con Línea de Visión (LOS).

- **Tiempo de Arribo (TOA):** el tiempo que pasa desde que la fuente emite la señal hasta que se recibe; deberemos estar sincronizados con la fuente. Permite calcular la distancia con mucha exactitud. Se utiliza, por ejemplo, en los sistemas de posicionamiento satelital (GPS) y en métodos de localización de terminales en redes celulares.
- **Diferencia de Tiempo de Arribo (TDOA):** el lapso entre la recepción de la señal en distintos sensores y/o nodos. No es indispensable sincronizarse o conocer la fuente, pero sí deben estar sincronizados los receptores y conocerse con exactitud sus ubicaciones. Presenta dificultades con fuentes móviles porque sus modelos matemáticos asumen una fuente estática, el cambio de frecuencia percibida por el efecto Doppler afecta la estimación.
- **Diferencia de Frecuencia de Arribo (FDOA):** mismo concepto que TDOA pero aplicado a la frecuencia percibida de la señal; en este caso aprovecha el efecto Doppler para trabajar específicamente con fuentes en movimiento.
- **Potencia de Señal Recibida (RSS):** compara la potencia medida en cada sensor y, con un modelo de propagación, estima la distancia a la fuente. Es de relativamente menor precisión y depende mucho de que el modelo se ajuste bien al entorno, pero es más sencillo de aplicar.

Si contamos con múltiples nodos, una vez adquiridos los parámetros podemos obtener la ubicación tridimensional de la fuente aplicando las siguientes técnicas (Fig. 2.5):

- **Trilateración:** habiendo obtenido la distancia a la fuente, por ejemplo con TOA, se pueden trazar en cada nodo círculos o esferas que tengan esta distancia como radio y la ubicación de la fuente será su intersección. Se necesitan por lo menos tres nodos para estimación 2D y cuatro para estimación 3D.
- **Hipérbolas:** el TDOA de un par de nodos nos indica la distancia relativa a ambos, podemos usar esto para trazar hipérbolas donde los nodos son los focos que la definen. La localización nuevamente será en la intersección, por lo que necesitamos por lo menos tres nodos.
- **Triangulación:** la fuente estará en la intersección de distintas direcciones de arribo. Requerirá por lo menos dos DOA en acimut y dos en elevación

para una localización en tres dimensiones. Su precisión dependerá de la cantidad de DOA que se tengan y de la precisión individual de cada una.

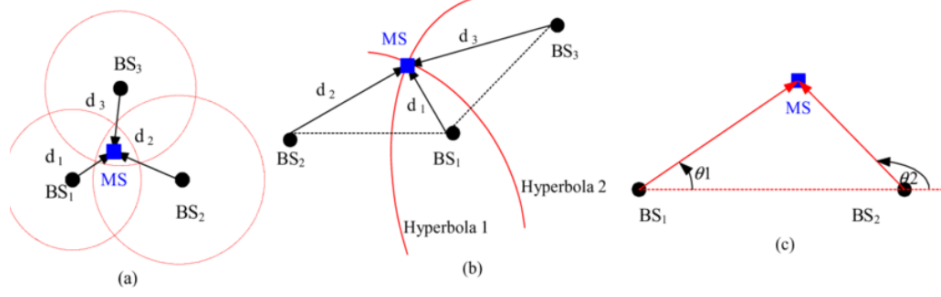


Figura 2.5: Trilateración (a), hipérbolas (b) y triangulación (c) [Zaidi, 2010].

En orden de extraer estos parámetros de las señales acústicas se utilizan distintos algoritmos. Como en este trabajo se utilizaron los algoritmos GCC-PHAT y MUSIC para estimar la Dirección de Arribo, desarrollaremos estos dos conceptos en las siguientes secciones. Se puede encontrar más información sobre TOA, TDOA y FDOA en (Xu et al., 2011; Li et al., 2016); se desarrolla sobre algoritmos de estimación de DOA diferentes de MUSIC en (Chen et al., 2010; Friedlander, 2009; Van Trees, 2001, 2002); en (Cobos et al., 2017; Jekaterýńczuk & Piotrowski, 2023) se hace un estudio general de los principales métodos utilizados en SSL, incluyendo IA.

2.4. Estimación de Dirección de Arribo

Extraer el ángulo de arribo de una señal incidente es el proceso central de este trabajo. En esta sección examinaremos en qué consiste el proceso en general y en nuestro caso particular: sus fundamentos, características y aspectos prácticos. Se dará una clasificación a diferentes tipos de algoritmo posibles, se justificará la elección de los algoritmos GCC-PHAT y MUSIC y se los explicará en más detalle.

2.4.1. Estudio general

Como fue mencionado previamente, la Dirección de Arribo es uno de los parámetros de la señal que podemos utilizar para determinar la ubicación de la fuente. Consiste en una o las dos dimensiones angulares del sistema de coordenadas esférico (acimut y/o elevación). En algunos casos no es relevante o necesario conocer la distancia, por lo que la DOA es todo lo que se requiere para ubicar la fuente.

La localización utilizando un arreglo de antenas se basa fundamentalmente en el hecho de que cada elemento del arreglo recibirá una señal levemente diferente

a las demás: identificando e interpretando estas diferencias podemos estimar la dirección de la señal.

Comenzaremos modelando el escenario. Consideremos I fuentes que emiten I señales por un medio isotrópico y lineal, lo que quiere decir que las propiedades físicas son constantes en todas direcciones y las señales se pueden superponer de forma lineal. Asumimos que estamos en campo lejano y que las señales son de frecuencia única o banda angosta: la distancia es lo suficientemente grande para que los frentes de onda de las señales sean planos y el contenido en frecuencia se concentra alrededor de una frecuencia central f_c .

Las señales inciden sobre un Arreglo Uniforme Lineal (ULA) donde hay M sensores ubicados en una línea recta, cada uno a la misma distancia Δ del siguiente (Fig. 2.6). Al impactar con cierto ángulo θ tendrán que recorrer un camino más largo para llegar a los elementos más lejanos, y esta distancia (o tiempo) extra se traduce en una diferencia de fase en la señal recibida. Si llamamos s_{i1} a la señal recibida por el primer elemento, entonces la señal en el elemento m será

$$s_{im}(t) = s_{i1}(t) e^{j(m-1)\mu_i} \quad (2.7)$$

donde

$$\mu_i = -\frac{2\pi}{\lambda} \Delta \sin(\theta_i) \quad (2.8)$$

es la denominada frecuencia espacial que le corresponde a cada ángulo de incidencia. El objetivo de la estimación de DOA es extraer esta frecuencia espacial de las señales recibidas en el arreglo.

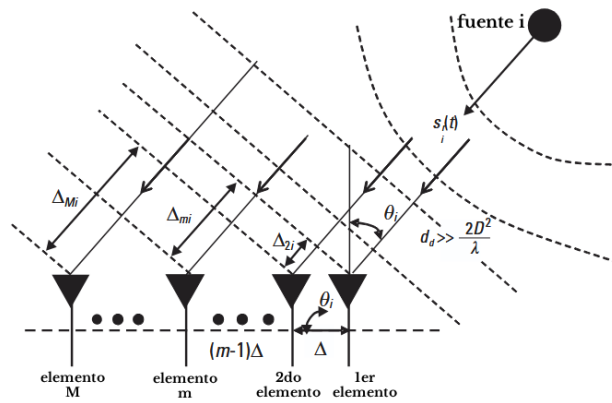


Figura 2.6: Señal incidiendo en un ULA de M elementos [Chen, 2010].

La señal recibida completa puede representarse de la siguiente manera:

$$\mathbf{x}(t) = [\mathbf{a}(\mu_1) \cdots \mathbf{a}(\mu_I)] [s_1(t) \cdots s_I(t)]^T + \mathbf{n}(t) = \mathbf{A} \mathbf{s}(t) + \mathbf{n}(t) \quad (2.9)$$

donde $\mathbf{x}(t)$ es un vector de las señales recibidas por los M elementos del arreglo, $\mathbf{s}(t)$ es la parte “pura” de la señal, $\mathbf{n}(t)$ es el vector de ruido AWGN y $\mathbf{A}$ ($M \times I$) es una matriz conocida que contiene los vectores de steering o dirección: funciones que representan las características espaciales del arreglo, cómo respondería cada elemento a cada ángulo de llegada. Las matrices de steering dependen de la configuración de cada arreglo y, por ejemplo, para un ULA sus columnas se ven de la siguiente forma:

$$\mathbf{a}(\mu_i) = \begin{bmatrix} 1 & e^{j\mu_i} & e^{j2\mu_i} & \dots & e^{j(M-1)\mu_i} \end{bmatrix}^T \quad (2.10)$$

Se puede observar cómo (2.8) define los desplazamientos de fase para cada elemento en función de μ_i , es decir, según su ubicación respecto a los demás elementos y el ángulo de arribo; esto es verdadero para los vectores de steering de cualquier arreglo.

Con la señal $\mathbf{x}(t)$ se pueden utilizar distintos métodos para estimar la ubicación de la fuente, que se pueden clasificar en:

1. **Basados en TDOA:** miden primero la Diferencia de Tiempo de Arribo y a partir de ella calculan la DOA de forma geométrica. Son, comparados a los demás métodos, de bajo costo computacional y complejidad, mientras que su limitación está en su baja resolución, sobre todo para arreglos compactos. El algoritmo GCC-PHAT, utilizado en el primer sistema del proyecto, pertenece a este grupo.
2. **Beamforming:** consiste en construir o “moldear” el patrón de los transductores. Se construye un haz sensible a una dirección y se realiza un barrido con él en todas direcciones para determinar cuál es la correcta. Comenzando desde el concepto más simple como el retardo y suma (Delay and Sum, DS), se puede ir complejizando con, por ejemplo, el método de Respuesta de Mínima Varianza sin Distorsión (MVDR).
3. **Subespacios:** también llamados de alta resolución, no realizan un barrido con un haz sino que analizan la estructura algebraica de la matriz de covarianza de las señales del arreglo. Es la base del algoritmo MUSIC utilizado en el segundo sistema. La idea fundamental es que se puede descomponer la matriz de covarianza en dos subespacios ortogonales: un subespacio de señal asociado a las contribuciones de la fuente y un subespacio de ruido. Su costo computacional es mayor pero es aplicable en tiempo real para arreglos de pocos elementos en hardware embebido como el utilizado en este trabajo.
4. **Máxima Verosimilitud (ML):** métodos paramétricos que buscan recons-

truir la señal deseada a partir de la señal recibida. Pueden lograr el mejor rendimiento teórico, especialmente en bajo SNR, pero son computacionalmente muy complejos (búsqueda multidimensional), por lo que no son prácticos en tiempo real.

Estos son los principios y modelos en los que se basan la mayoría de los algoritmos clásicos de estimación de DOA. Se resumen en la Tabla 2.1.

Tabla 2.1: Tipos de algoritmos clásicos de estimación de DOA.

Tipo	Descripción	Ventajas y desventajas	Ejemplos
Basados en TDOA	Primero miden TDOA y con eso calculan DOA	(+) Simplicidad, bajo costo computacional. (-) Baja resolución, sensible a ruido	GCC-PHAT
Beamforming	Dirigen el haz del arreglo y miden la potencia en cada dirección	(+) Bajo costo, robustez, mejorable. (-) Baja resolución, sensible a interferencias	Beamscan, Delay-and-Sum, MVDR
Subespacios	Aprovechan la descomposición en subespacios ortogonales de señal y ruido	(+) Alta resolución, múltiples fuentes. (-) Sensible a interferencias correlacionadas, costo alto	MUSIC, ESPRIT
Máxima Verosimilitud	Reconstruyen la señal deseada con los parámetros más precisos	(+) Mejor rendimiento en baja SNR. (-) Muy intensivo, sensible a desajustes del modelo	ML, MAP, LS

2.4.2. Algoritmo GCC-PHAT

El primero de los algoritmos que desarrollaremos en profundidad es la Correlación Cruzada Generalizada con Transformada de Fase o GCC-PHAT. Es un buen punto de partida para comprender la estimación de DOA y uno de los más sencillos de aplicar, razón por la cual se utilizó en el primer modelo construido en este trabajo.

GCC fue presentado en (Knapp & Carter, 1976) donde postulan que el trabajo principal es estimar el retardo temporal entre la recepción de una señal en dos sensores separados (TDOA), ya que esta está relacionada geoméricamente con la dirección de arribo. El método que utilizan es el de la función de correlación (o covarianza) cruzada entre las dos señales: miden el grado de parecido entre una señal y una versión desplazada de la otra, siendo la variable de la función este desplazamiento. Nuestra estimación será el valor que maximice la función de correlación. Según el modelo de señal presentado, para dos señales impactando sobre dos sensores separados, se vería así:

$$x_1(t) = s(t) + n_1(t) \quad (2.11)$$

$$x_2(t) = \alpha s(t - \tau_0) + n_2(t) \quad (2.12)$$

$$r_{12}(l) = \mathbb{E}[x_1(t) x_2(t - l)] \quad (2.13)$$

donde n_1 y n_2 son realizaciones independientes del ruido en cada sensor.

Una limitación de la correlación cruzada, al estar en el dominio del tiempo, es que solo puede estimarse sobre un fragmento finito de señal, aumentando la probabilidad de error. Otro problema es que su “pico”, en donde se encuentra el máximo, puede ser ancho y poco definido.

La solución propuesta es, a través de una Transformada de Fourier, utilizar el equivalente en frecuencia a la correlación cruzada, el espectro cruzado de potencia (2.16): mientras el ruido de ambos canales no esté correlacionado, el espectro cruzado es esencialmente el espectro de potencia de la señal multiplicado por una exponencial compleja que contiene el retardo.

$$X_1(f) = S(f) + N_1(f) \quad (2.14)$$

$$X_2(f) = \alpha S(f) e^{-j2\pi f\tau_0} + N_2(f) \quad (2.15)$$

$$G_{12}(f) = \mathbb{E}[X_1(f) X_2^*(f)] \quad (2.16)$$

$$= |S(f)|^2 \alpha e^{j2\pi f\tau_0} \quad (2.17)$$

La idea de la correlación cruzada generalizada es realizar un filtrado antes de correlacionar para acentuar las frecuencias con SNR alto y así obtener estimaciones más precisas. En el dominio de la frecuencia esto se ve como una multiplicación por una función de ponderación $\Psi(f)$. La ponderación PHAT es

$$\Psi_{\text{PHAT}}(f) = \frac{1}{|G(f)|} \quad (2.18)$$

$$\text{GCC-PHAT}(f) = \frac{G(f)}{|G(f)|} = e^{j2\pi f\tau_0} \quad (2.19)$$

Al dividir el espectro por su propio módulo lo normalizamos: queda con módulo unitario y solo permanece la información de la fase. Como la información del retardo está en la fase (en la exponencial compleja que multiplica a la señal), al volver al dominio del tiempo el resultado es, idealmente, una delta perfecta en la posición del retardo.

Naturalmente, en la práctica los resultados no son ideales: el espectro cruzado con el que trabajamos no es exacto, y la ponderación PHAT, al normalizar el espectro, considera a todas las frecuencias por igual, las que contienen información de la señal y las que son puro ruido, y estas últimas introducen errores. Es por esto que se debe acotar el análisis a la banda donde efectivamente esté la señal.

2.4.3. Algoritmo SRP-PHAT

Habiendo establecido el funcionamiento de GCC-PHAT para un par de micrófonos, podemos definir a SRP-PHAT (Steered Response Power - Phase Transform) como la generalización de GCC-PHAT para arreglos de múltiples sensores. El avance conceptual de este algoritmo es pasar de calcular el TDOA entre dos sensores a evaluar la potencia de salida de un filtro espacial o beamformer: se escanea una grilla espacial de posibles direcciones de arriba y se evalúa en cada punto cuánta “energía direccional” recibiría si la fuente proviniera de ahí:

1. Se calcula el retardo teórico que tendría una señal desde cada dirección en cada micrófono como

$$\tau_i(\varphi, \theta) = -\frac{r_i \hat{d}(\varphi, \theta)}{v} \quad (2.20)$$

$$\hat{d}(\varphi, \theta) = [\sin(\varphi) \cos(\theta), \cos(\varphi) \cos(\theta), \sin(\theta)] \quad (2.21)$$

donde r_i es la posición relativa del micrófono i dentro del arreglo y $\hat{d}(\varphi, \theta)$ un vector unitario que representa la dirección de arriba, ambas en coordenadas cartesianas.

2. Se calcula la TDOA teórica que tendría cada combinación de dos micrófonos (i, j) para cada dirección (φ, θ) .

$$\tau_{i,j}(\varphi, \theta) = \tau_i(\varphi, \theta) - \tau_j(\varphi, \theta) \quad (2.22)$$

3. Se calcula la potencia recibida de cada dirección como la sumatoria de GCC-PHAT para cada par de micrófonos evaluado en los retardos teóricos de esa

dirección, o sea, qué tanto se parecen los retardos de la señal recibida con los teóricos. La dirección estimada será aquella donde esta suma sea máxima.

$$\text{SRP}(\varphi, \theta) = P(\varphi, \theta) = \sum_{i,j} \text{GCC-PHAT}_{i,j}(\tau_{i,j}(\varphi, \theta)) \quad (2.23)$$

Este algoritmo se presenta no solo porque se puede ver como una evolución de GCC-PHAT, sino también porque se implementa en el sistema de cuatro micrófonos: puede funcionar como respaldo para MUSIC en la localización de drones y, lo que es más relevante, como el algoritmo mejor dotado para la localización de detonaciones de artillería u otras fuentes no sostenidas de banda ancha: la ponderación PHAT normaliza el espectro por lo que las señales de banda más ancha generan picos más estrechos, y no requiere acumular observaciones para condicionarse, por lo que puede ofrecer mejores estimaciones en tiempos muy cortos. Se analizan con detalle las razones, ventajas y desventajas de cada uno en la Sección 4.2.3.

2.4.4. Algoritmo MUSIC

MUSIC (Multiple Signal Classification, Clasificación de Señales Múltiples) es el otro algoritmo utilizado en este proyecto. Presentado en (Schmidt, 1986), es uno de los métodos de estimación de DOA por subespacios más utilizados en el procesamiento de arreglos, y de su idea central han surgido múltiples variantes.

Regresando al modelo de señal para un arreglo (ec. 2.9), se tiene un vector $\mathbf{s}$ de señal pura multiplicada por una matriz $\mathbf{A}$ de dirección o steering que contiene la respuesta del arreglo a los ángulos en que puede incidir la señal, y un vector $\mathbf{n}$ de ruido aditivo.

La hipótesis del método es que, cuando una señal incide sobre algún elemento de un arreglo, la parte de la señal pura tiene correlación con lo recibido en otros elementos ya que proviene de la misma fuente, mientras que la parte del ruido, si efectivamente es AWGN, no tendrá correlación ni con las componentes de la señal ni con el ruido de los demás elementos. Para aprovechar esto se calcula una matriz de covarianza espacial:

$$\mathbf{R}_{xx} = \mathbb{E}\{\mathbf{x}(t) \mathbf{x}^H(t)\} = \mathbf{A} \mathbf{R}_{ss} \mathbf{A}^H + \sigma_n^2 \mathbf{I} \quad (2.24)$$

donde $\mathbb{E}\{\cdot\}$ denota la esperanza estadística y $(\cdot)^H$ es el operador hermítico (con-

jugar y transponer). Para poder computarla se aproxima de la siguiente forma:

$$\mathbf{R}_{xx} \approx \hat{\mathbf{R}}_{xx} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}(t_n) \mathbf{x}^H(t_n) = \frac{1}{N} \mathbf{X} \mathbf{X}^H \quad (2.25)$$

donde $\mathbf{X}$ es la matriz de la señal con ruido compuesta por N snapshots o muestreos instantáneos (ec. 2.26); mientras mayor sea N será una mejor aproximación pero tardará más tiempo.

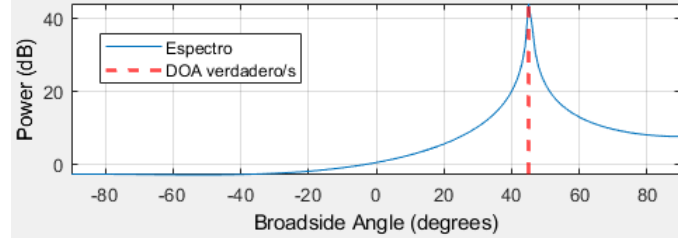
$$\mathbf{X} = \begin{bmatrix} \mathbf{x}(t_1) & \mathbf{x}(t_2) & \cdots & \mathbf{x}(t_N) \end{bmatrix} \quad (2.26)$$

Al ser una matriz hermítica, se puede descomponer $\mathbf{R}_{xx}$ en M autovalores y autovectores (eigendecomposition, Apéndice A). Los autovalores representan la magnitud de covarianza de los datos, por lo que los I autovalores más grandes corresponden a las I señales (el subespacio de la señal) y los $M - I$ restantes corresponden al subespacio del ruido. Es en este momento cuando se puede estimar el número de fuentes, ya que suele haber una diferencia notable en la magnitud de los autovalores correspondientes a la señal y el ruido.

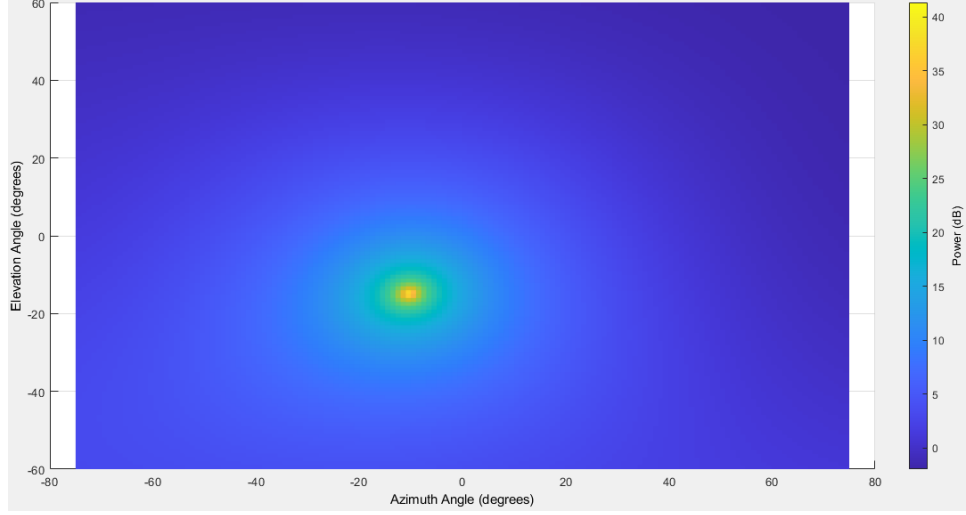
La clave en los algoritmos de subespacio como MUSIC es que los subespacios de señal y de ruido son ortogonales entre sí: el vector de dirección de cada fuente es ortogonal a todo el subespacio de ruido. Se calcula el subespacio de ruido a partir de la matriz de covarianza medida y se escanean los vectores de dirección para observar cuál/cuáles son ortogonales. Si llamamos $\mathbf{a}(\varphi, \theta)$ al vector de dirección $\mathbf{a}(\mu_i)$ evaluado en los ángulos de acimut φ y elevación θ , y $\mathbf{V}_n$ a la matriz cuyas columnas son los autovectores del subespacio de ruido, el pseudoespectro MUSIC se define como el inverso de la norma al cuadrado de la proyección del vector de dirección sobre el subespacio de ruido:

$$P_{\text{MUSIC}}(\varphi, \theta) = \frac{1}{\mathbf{a}^H(\varphi, \theta) \mathbf{V}_n \mathbf{V}_n^H \mathbf{a}(\varphi, \theta)} = \frac{1}{\|\mathbf{V}_n^H \mathbf{a}(\varphi, \theta)\|^2} \quad (2.27)$$

Cuando (θ, ϕ) coincida con la dirección de una fuente, la proyección será cero y en el pseudoespectro se verá un pico; las ubicaciones de los I picos más altos corresponden con las direcciones de las I fuentes. En la Fig. 2.7 se presentan ejemplos de pseudoespectros para una y dos dimensiones.



(a) Pseudoespectro unidimensional.



(b) Pseudoespectro bidimensional.

Figura 2.7: Ejemplos de pseudoespectros MUSIC para una y dos dimensiones.

El pseudoespectro se proyecta utilizando el subespacio de ruido, y no el de la señal, porque en la dirección verdadera tiende a cero y su inversa genera un pico más agudo que el que se obtiene buscando el máximo con el subespacio de señal. Cabe aclarar que el subespacio de ruido y el de señal contienen la misma información, son complementos ortogonales, por lo que el estimador se puede calcular indistintamente con cualquiera de los dos: si estimamos el subespacio de señal utilizaremos su complemento para el pseudoespectro. En la Sección 4.2.2 se desarrolla como se aprovechó esta propiedad para reducir el costo de cómputo.

El algoritmo descrito es el original que ha dado lugar a numerosas variantes, se mencionan Root-MUSIC (reformula la búsqueda de picos como la búsqueda de las raíces de un polinomio, aplicable en arreglos lineales uniformes), ESPRIT (explota una estructura de invarianza del arreglo, necesita arreglos descomponibles en sub-arreglos simétricos) y variantes que combinan MUSIC con algoritmos de optimización.

El algoritmo original es un método de banda angosta, supone idealmente una sola frecuencia para que el vector de dirección esté bien definido. Pero las señales de este proyecto, las generadas por los drones, son de banda ancha, y el vector de dirección depende de la frecuencia. Para aplicar MUSIC, u otros algoritmos

de banda angosta, a señales de banda ancha hay dos estrategias principales, conocidas como incoherente y coherente (Brandstein & Ward, 2001):

- **Incoherente:** se descompone la banda ancha en múltiples bandas angostas (bins) y se aplica MUSIC en cada una, estimando múltiples matrices de covarianza y subespacios de ruido. Los pseudoespectros de cada banda se promedian para llegar a un resultado único. Es el método más directo y simple, conviene utilizarlo cuando se requiere velocidad de procesamiento o cuando hay bins de frecuencias con SNR especialmente alto porque permite seleccionarlos o ponderarlos con mayor peso para la estimación.
- **Coherente:** *Coherent Signal Subspace Method* (CSSM) intenta combinar la información de todas las bandas en una única matriz de covarianza, transformando las matrices para mapearlas o “afinarlas” a una frecuencia central, antes de realizar la descomposición en autovalores. Al contar con *snapshots* de todos los bins para calcular $\mathbf{R}_{xx}$ se tiene más información para estimar los subespacios, consigue distinguir mejor múltiples fuentes o ruidos correlacionados

El sistema de este proyecto utiliza MUSIC de banda ancha incoherente. Su simplicidad permite estimar la DOA en tiempo real utilizando hardware de bajo costo, y el comportamiento armónico de la señal del dron (analizado en la sección 4.2) permite implementar una ponderación de bins según su SNR. Implementar MUSIC coherente puede, en teoría, mejorar el rendimiento en escenarios de múltiples fuentes o con mucha reverberación, pero no fue probado en este trabajo.

Algoritmo implementado: MUSIC de banda ancha incoherente con escaneo bidimensional

Implementar MUSIC en el sistema diseñado requiere adaptar el algoritmo a las condiciones propias del sistema: las señales, el muestreo, las capacidades del hardware. Se presenta a continuación un diagrama en bloques del algoritmo utilizado y una descripción del funcionamiento de cada etapa.

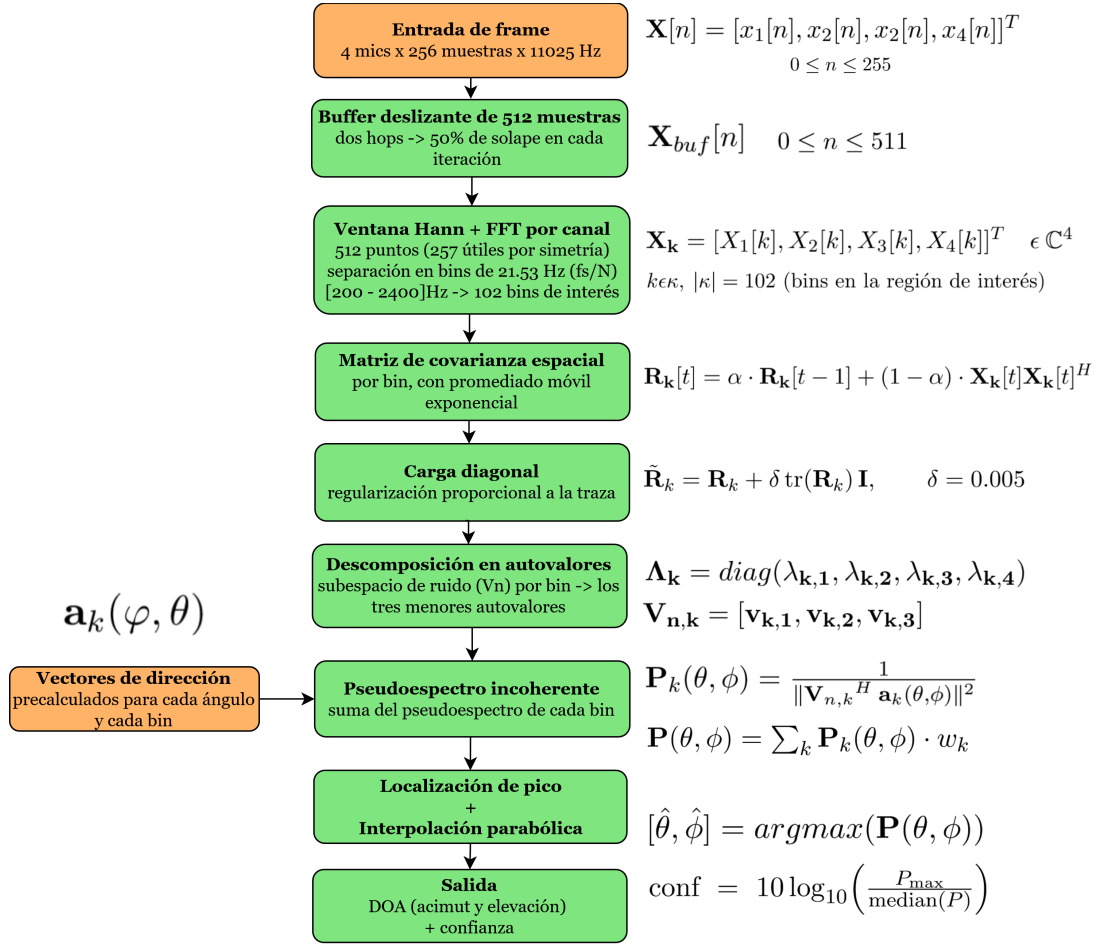


Figura 2.8: Diagrama en bloque del algoritmo MUSIC implementado.

1. Los vectores de dirección se precálculan una única vez y se guardan para calcular los pseudoespectros en cada *frame*. Se calculan para cada punto de la grilla y cada bin de frecuencia como

$$\mathbf{a}_k(\varphi, \theta)[m] = e^{-j2\pi f_k \tau_m} \quad (2.28)$$

$$\tau_m = \frac{\mathbf{r}_m \cdot \hat{\mathbf{d}}}{c} \quad (2.29)$$

$$\hat{\mathbf{d}} = [\sin \varphi \cos \theta \quad \cos \varphi \cos \theta \quad \sin \theta] \quad (2.30)$$

siendo τ_m el retardo de propagación, r_m la posición del micrófono m , y $\hat{\mathbf{d}}$ el vector unitario hacia la fuente.

2. **Entrada de hops:** la captura de datos es ajena al algoritmo y se describe en la Sección 4.2, pero se menciona que los datos llegan en bloques de 256 muestras por canal, muestreadas a 11025 Hz. Estos valores son configurables.
3. **Buffer deslizante:** mantiene las últimas 512 muestras de cada canal (dos

hops, 50 % de solapamiento en cada análisis) para aumentar la longitud de la FFT y obtener mayor resolución espectral sin tener que aumentar el tamaño de los *hops* ni la latencia.

4. **Ventana de Hann y FFT por canal:** multiplica el *buffer* por una ventana de Hann periódica para reducir la fuga espectral y calcula una RFFT (Transformada Rápida de Fourier para datos Reales, no calcula los valores de frecuencias negativas). Aplicar la misma ventana a todos los canales no modifica las diferencias de fase entre micrófonos, por lo que no altera la estimación de DOA. Al tener 512 puntos de entrada (las muestras) a una frecuencia de muestreo de 11 kHz, el tamaño de los 257 'bins' de frecuencia (la salida de la FFT) es de $f_s/N_{FFT} = 21,53Hz$, de los 257 se toman solamente los 102 que pertenecen a la Región de Interés (ROI) de 200-2400 Hz.
5. **Matriz de covarianza espacial:** para cada bin de la ROI se calcula y mantiene una matriz propia, actualizada con **promediado móvil exponencial**. Que tenga 'memoria' es necesario para que R_k esté definida correctamente y que los picos del pseudoespectro sean más nítidos ya que, al ejecutar el algoritmo en tiempo real, el vector X_k que utilizamos para calcularlo cuenta con solo un nuevo valor complejo para cada sensor por iteración.

$$\mathbf{R}_k[t] = \alpha \cdot \mathbf{R}_k[t-1] + (1 - \alpha) \cdot \mathbf{X}_k[t]\mathbf{X}_k[t]^H \quad (2.31)$$

Aumentar el α le da más estabilidad a R_k , pero disminuye la capacidad de estimar nuevos DOAs (eventos cortos, fuentes en movimiento).

6. **Carga diagonal:** es una técnica que consiste en sumar una pequeña constante (el 'factor de carga' δ) a la diagonal de la matriz de covarianza para que sea numéricamente más estable y prevenir la singularidad de R_k ante malas estimaciones (Van Trees, 2002). Es proporcional a la traza (la suma de los elementos de la diagonal) de cada bin para que cada uno reciba una carga acorde a su propia potencia. Al desplazar todos los autovalores por igual no modifica el subespacio de ruido y, por lo tanto, el pseudoespectro.

$$\tilde{\mathbf{R}}_k = \mathbf{R}_k + \delta \text{tr}(\mathbf{R}_k) \mathbf{I}, \quad \delta = 0,005 \quad (2.32)$$

7. **Descomposición en autovalores:** desarrollado en el Apéndice A, aprovecha las propiedades de la matriz de covarianza para estimar el subespacio del ruido.
8. **Pseudoespectro incoherente:** con los vectores de dirección precalculados, se computa el pseudoespectro de cada bin de la ROI con la ec. 2.27 y

se suman para obtener el pseudoespectro de toda la banda de trabajo.

9. **Localización de pico e interpolación parabólica:** se toma el *argmax* del pseudoespectro como la primera estimación y luego se refina con una interpolación parabólica. La interpolación parabólica consiste en ajustar una parábola a los tres valores del pseudoespectro en torno al pico (el del pico y los dos adyacentes) y tomar el pico de esta curva. De esta forma se puede tener una resolución sub-grilla (Fig. 2.9).

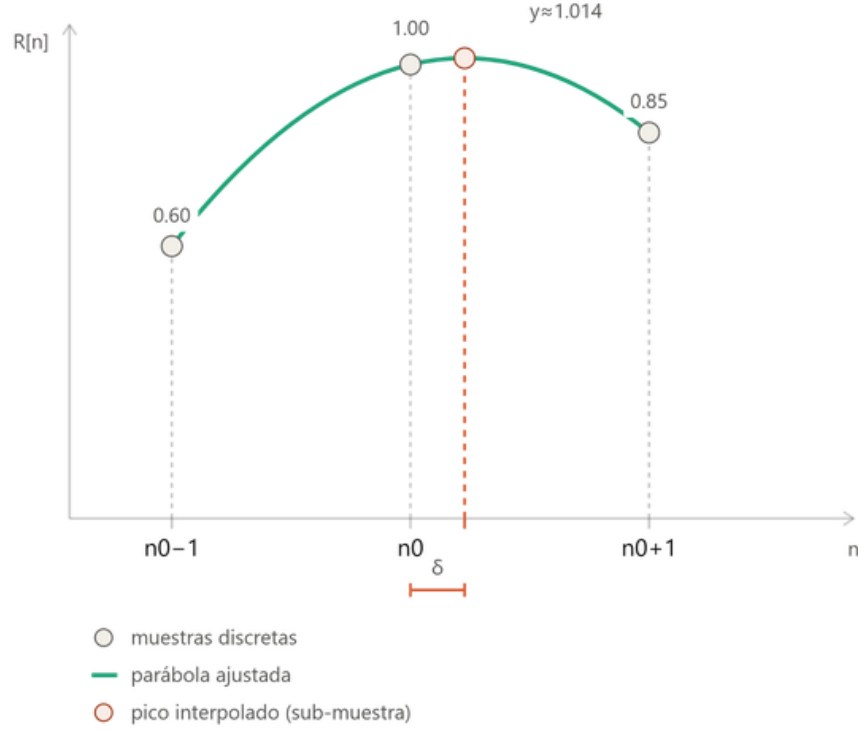


Figura 2.9: Interpolación parabólica.

10. **Salida:** los valores que entrega el algoritmo a su salida son la estimación bidimensional de DOA $[\varphi, \theta]$, el pseudoespectro completo y la confianza de la estimación en dB , calculada como el logaritmo entre la potencia del pico sobre la mediana del pseudoespectro. La confianza cuantifica la nitidez del pico estimado, permite desechar estimaciones imprecisas para que, por ejemplo, no se registren o no se sigan con los servomotores.

Como el arreglo no mide directamente el ángulo de arribo de la señal sino el retardo entre sensores, es conveniente que la grilla de escaneo sea de la misma naturaleza: en lugar de escanear uniformemente en grados se escanea en el espacio- u , el cual captura la proyección de la señal sobre el arreglo, o cómo varía la fase (Van Trees, 2002). Para un arreglo planar en XZ donde el acimut φ y la elevación θ se miden respecto a la normal (el eje Y) se define los cosenos directores u_x y u_z , donde el retardo es lineal

$$u_x = \sin(\varphi)\cos(\theta) \quad (2.33)$$

$$u_z = \sin(\theta) \quad (2.34)$$

El retardo medido en cada sensor m , ubicado en (x_m, z_m) dentro del arreglo, será

$$\tau_m = -\frac{x_m u_x + z_m u_z}{c} \quad (2.35)$$

La grilla del estimador avanza uniformemente en el *seno de cada ángulo* esférico ($\sin(\varphi)$ y $\sin(\theta)$), no en los cosenos directores. En elevación coincide exactamente con u_z , pero en acimut difiere por un factor $\cos(\theta)$: a medida que la elevación se aleje de 0° los puntos serán más densos a lo largo del eje u_x , se pierde eficiencia escaneando puntos muy cercanos en un lugar donde el arreglo tiene poca resolución y la proyección en ángulos se distorsiona. Es el mismo efecto que sucede cuando se proyecta el globo terráqueo a un mapa. Con el rango de elevación configurado a $\pm 35^\circ$ esto no afecta significativamente, pero en caso de aumentarlo se recomienda considerar modificar el modo de escaneo. Mapea todas las direcciones (φ, θ) a un círculo unitario $u_r = \sqrt{u_x^2 + u_z^2} \leq 1$. La proyección en el espacio angular concentra los puntos en el centro del arreglo, lo que se corresponde con el funcionamiento físico del arreglo (Fig. 2.10).

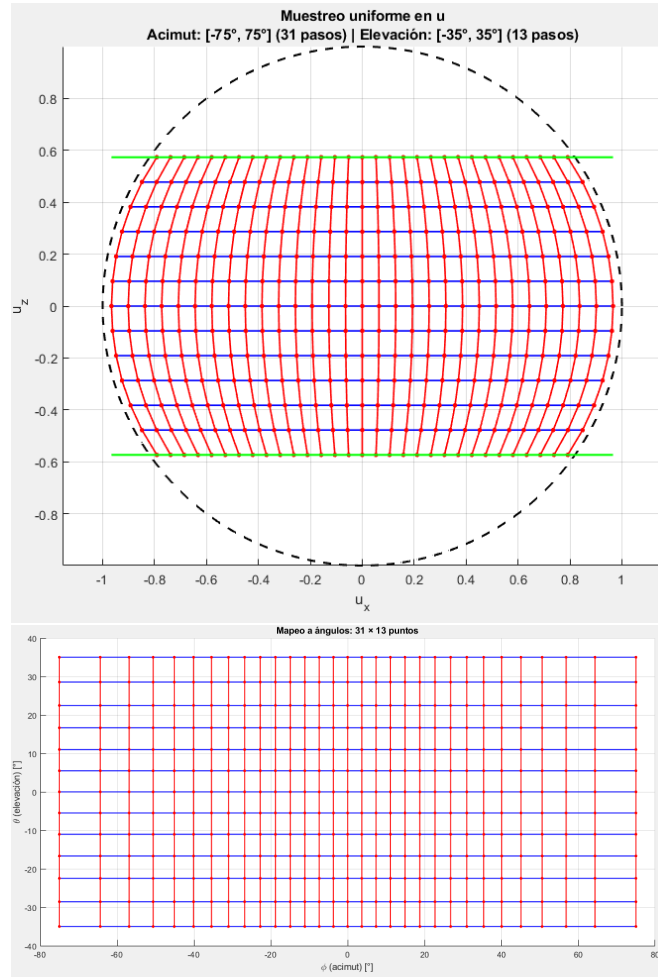


Figura 2.10: Proyección del espacio- u en el espacio angular. Rango y cantidad de puntos utilizados en el sistema de cuatro micrófonos

2.5. Análisis de sistemas similares para el diseño del sistema

Como parte del trabajo realizado en la PPS, se buscaron y estudiaron papers teóricos y diseños con características parecidas al propio para extraer información, ideas y recomendaciones. Dentro de un estudio más amplio, nos centramos en (Belloch et al., 2019; Case et al., 2008; Fabregat et al., 2020; Ramamonjy et al., 2018; Riabko et al., 2024; Zhang et al., 2014; Zhu et al., 2021) y sacamos las siguientes conclusiones:

1. **Definir el equilibrio entre precisión y costo:** antes de comenzar a diseñar, se debe saber cuánta precisión y robustez se requiere, y cuánto se está dispuesto a pagar por ellas. Esto determinará las opciones para armar el sistema.

2. **Considerar el escenario de uso:** si estará expuesto al clima, si habrá multipathing, si estará en una ubicación remota con dificultades de alimentación y/o transmisión, el tipo de señal con la que debe trabajar.
3. **Arreglos sencillos y con pocos micrófonos:** en escenarios como el planteado en este trabajo, donde no se requiere una localización exacta sino una detección y localización precisa pero dentro de un margen de error, que puede ser una primera línea de detección dentro de un sistema más completo y que no espera muchas fuentes simultáneas, el aumento de precisión que otorgan los arreglos numerosos y/o complejos geométricamente no suelen justificar el aumento de dificultad que conllevan. Naturalmente, si se cuenta con la capacidad de implementarlos, se obtienen mejores resultados y es posible localizar mayor cantidad de señales simultáneamente.
4. **Micrófonos:** el uso de micrófonos de alta calidad no implica necesariamente un incremento relevante de precisión frente a micrófonos más baratos con buen SNR. Por eso en muchos casos se utilizan micrófonos MEMS, que ofrecen buen rendimiento, digitalización y sincronización a bajo costo.
5. **Ningún algoritmo es universalmente superior** entre los clásicos y los basados en Machine Learning; se debe analizar caso por caso, idealmente utilizando enfoques híbridos.
6. **Firma acústica:** si se trabaja con drones, aprovechar su firma acústica conocida para utilizar principalmente señal útil.
7. En muchos de los casos vistos, la detección acústica es la primera línea disparadora dentro de un sistema más complejo.

2.6. Resumen del capítulo

En este capítulo se establecieron los fundamentos sobre los que se diseñan e implementan los sistemas del proyecto. Las señales de interés son señales acústicas modeladas como una superposición de sinusoides de distintas amplitudes, frecuencias y fases (ec. 2.2). Son señales de banda ancha, se trabaja específicamente entre 200 Hz y 2,4 kHz, por lo que distintas frecuencias arriban con desfases diferentes y el algoritmo elegido debe poder tolerarlo. Las longitudes de onda (ec. 2.4) asociadas limitan la geometría del arreglo debido al aliasing espacial, y existen fenómenos, principalmente la reverberación, que pueden desviar la dirección de arribo y/o generar interferencias.

Para el diseño del arreglo: aumentar la distancia entre micrófonos incrementa la resolución angular pero disminuye la frecuencia máxima sin ambigüedad; aumentar la cantidad de elementos mejora la precisión y la cantidad de señales distinguibles pero incrementa la complejidad. La sensibilidad es máxima en broadside y mínima en endfire, lo que justifica tanto el *tilt* (inclinación) del arreglo como la grilla de escaneo en espacio-u.

Entre los parámetros de localización posibles se elige, para el sistema principal, la Dirección de Arribo (DOA) ya que no exige sincronización con la fuente. Los tipos de algoritmos clásicos se compararon en la Tabla 2.1; de todos ellos, se implementa GCC-PHAT (basado en TDOA) en el primer sistema, y SRP-PHAT (beamforming) y MUSIC (subespacios) en el segundo. MUSIC es el algoritmo principal del sistema final: aprovecha las características algebraicas de la matriz de covarianza espacial (ec. 2.24), la descompone, estima el subespacio de ruido y proyecta los vectores de dirección sobre él; la dirección verdadera será ortogonal al ruido y su proyección será cero, lo que en el pseudoespectro (ec. 2.27) se verá como un pico. Como la señal es de banda ancha, se utiliza la variante incoherente.

Finalmente, la revisión de sistemas similares nos da información y recomendaciones para el diseño propio: arreglos sencillos y de bajo costo pueden alcanzar rendimientos suficientes, cercanos a soluciones más costosas, mientras se cuida el diseño y el escenario no sea extremo.

Capítulo 3

Simulaciones

Antes de diseñar e implementar un sistema propio, se realizaron simulaciones en Matlab con el objetivo de obtener información fundamental y familiarizarse con el funcionamiento de los sistemas de estimación de DOA. Se pudieron probar distintos algoritmos, configuraciones y parámetros para comparar su comportamiento antes de decidir cuál o cuales implementar. Las simulaciones no intentan replicar las condiciones exactas del escenario real (no se modela el sonido de un dron ni se aplican los algoritmos de banda ancha), sino que sirvieron como un acercamiento conceptual.

Primero se simuló el escenario más simple: un arreglo lineal uniforme (ULA) de dos sensores (Fig. 3.1) impactado por una señal (Fig. 3.2). Aquí ya se aprecian las diferencias de resolución entre los espectros de algunos algoritmos de estimación de DOA.

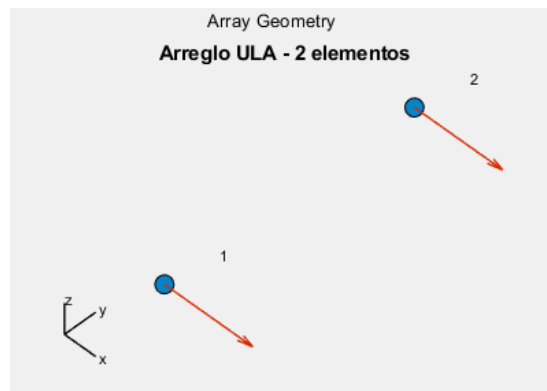


Figura 3.1: Arreglo ULA de dos elementos para simulación.

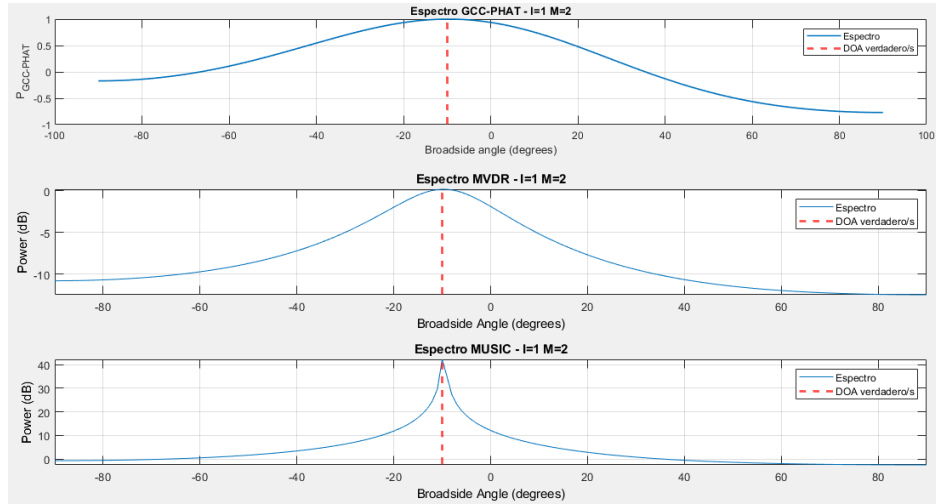


Figura 3.2: Pseudoespectros para distintos algoritmos en ULA de dos elementos.

A continuación se simulaban ULAs de más elementos donde se vieron mejoras en la resolución y, en los mejores casos (muchos elementos y/o algoritmos de mayor resolución como MUSIC), se pudieron distinguir más señales simultáneas ($I \leq M - 1$, Fig. 3.3). Se muestra una simulación con 20 elementos y dos señales a dos grados de distancia (3° y 5°).

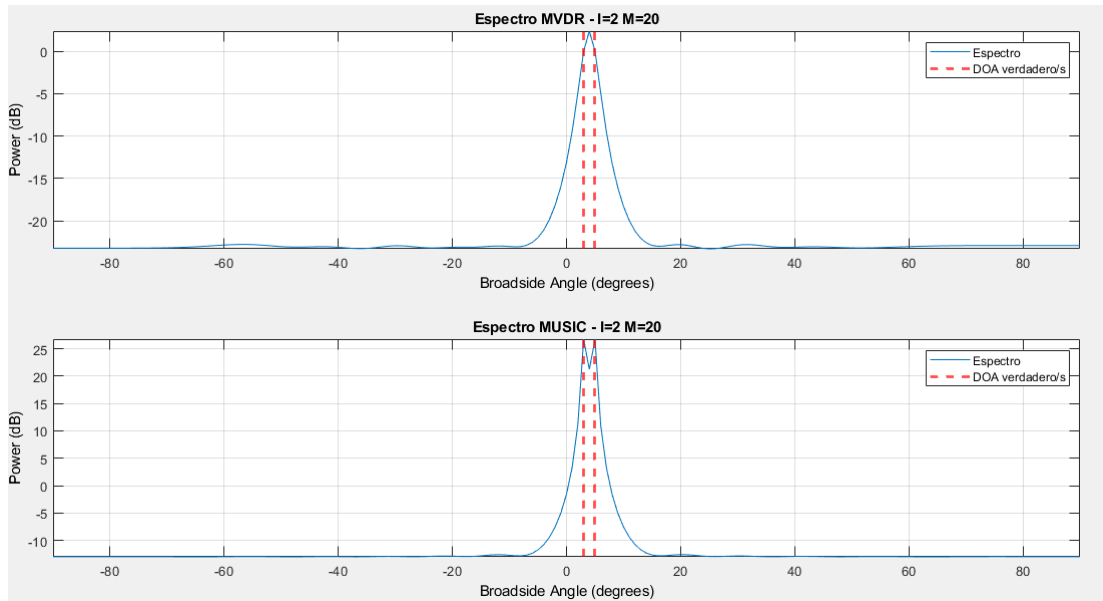


Figura 3.3: Resolución de MVDR y MUSIC para dos señales cercanas en ULA de 20 elementos.

Para ilustrar los efectos del aliasing espacial, simulamos un ULA de cuatro elementos impactado por dos señales donde la separación entre elementos es mayor a la mitad de la longitud de onda de la señal ($d > \lambda/2$, Fig. 3.4). Se aprecia la aparición de picos espurios que pueden causar una estimación errónea.

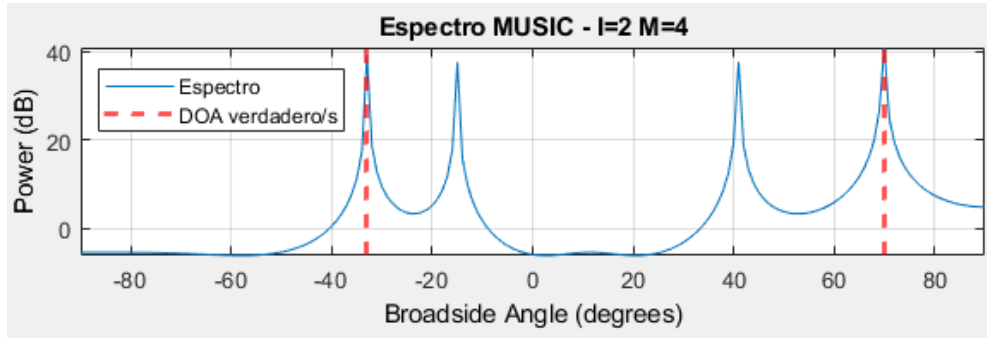


Figura 3.4: Efectos del aliasing espacial.

Luego se procedió a simular en dos dimensiones: acimut y elevación. Esto requiere simular arreglos planares o volumétricos; se eligieron un arreglo rectangular uniforme (URA) y un arreglo piramidal o tetraédrico, respectivamente. Se aplicó solamente el algoritmo MUSIC debido a que es el que ofrecía mayor resolución e ilustraba mejor las diferencias entre arreglos.

Viendo las siguientes figuras, se puede apreciar que el arreglo planar tiene un rango angular de 180° mientras que el volumétrico tiene un rango de 360° , la causa de esto es que el arreglo planar no tiene profundidad y no puede distinguir atrás de adelante, mientras que los arreglos volumétricos sí pueden y por eso tienen el rango completo. También se ve como en el arreglo rectangular la resolución es mejor en el centro del arreglo que en los bordes, esto se debe a que lo que en definitiva estiman estos sistemas es la diferencia de fase entre cada sensor, la cual viene dada por el seno del ángulo de arribo; cuando el ángulo es cercano a 0° , un pequeño cambio en dirección causa un cambio grande de fase, mientras que cerca de los 90° el cambio de fase es pequeño. El arreglo tetraédrico no tiene este problema porque, al tener mayor variedad geométrica, cuando la señal incide con ángulo cercano a los 90° para un par de micrófonos, hay otros pares para los cuales la DOA es más central, compensan la resolución.

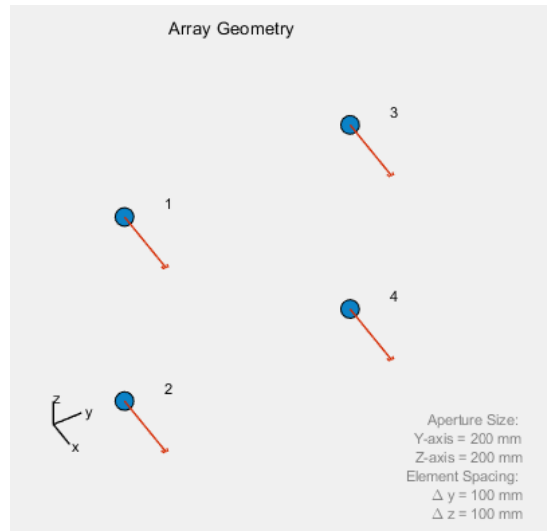
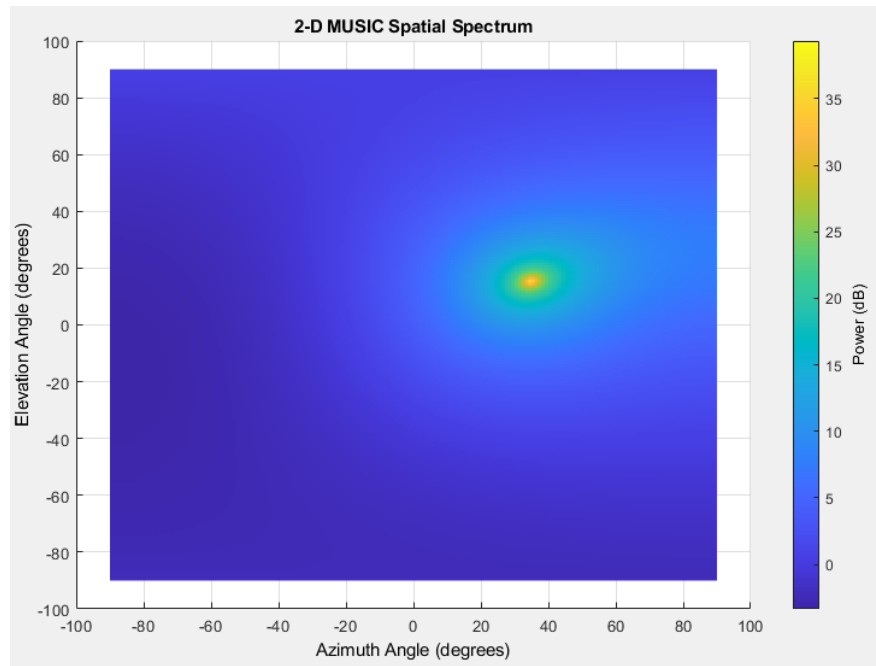


Figura 3.5: Arreglo URA de cuatro elementos para simulación.



(a) *Broadside.*

Figura 3.6: Resolución de MUSIC en *broadside* y *endfire* para URA de 4 elementos.

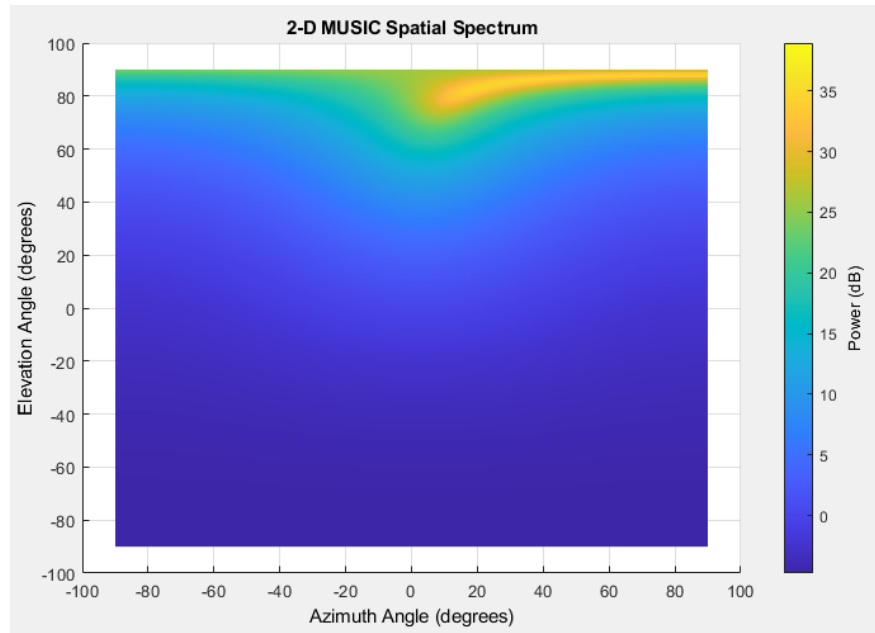
(b) *Endfire*.

Figura 3.6: Resolución de MUSIC en *broadside* y *endfire* para URA de 4 elementos (continuación).

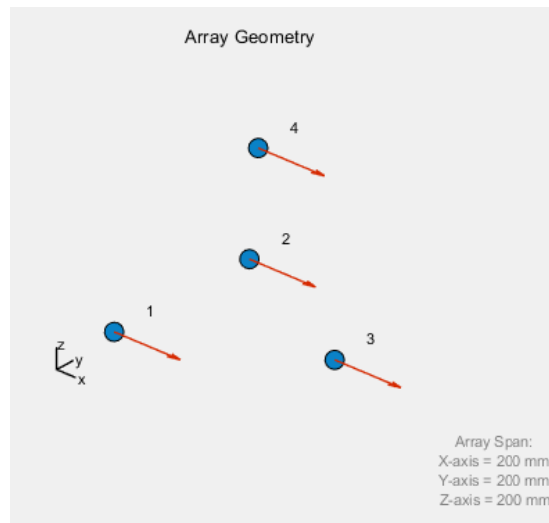


Figura 3.7: Arreglo tetraédrico de cuatro elementos para simulación.

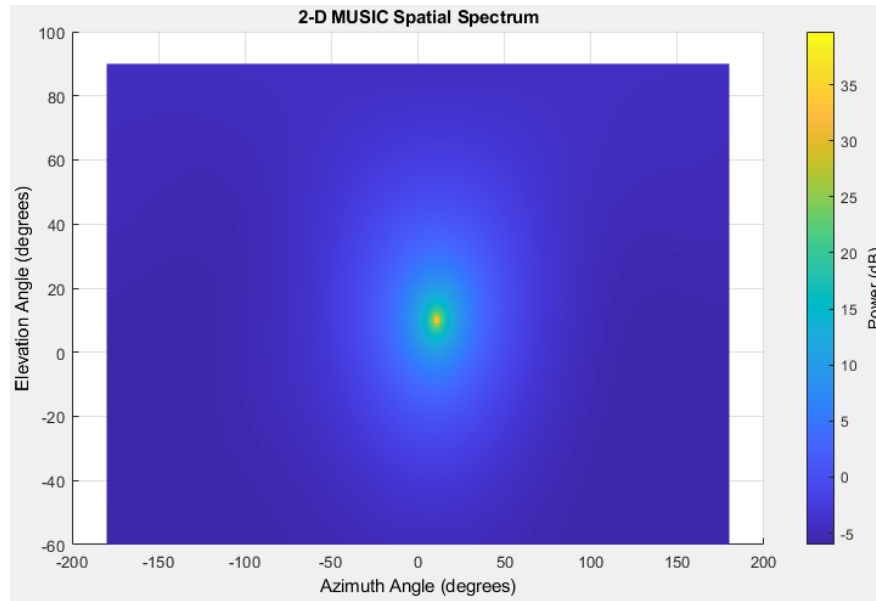
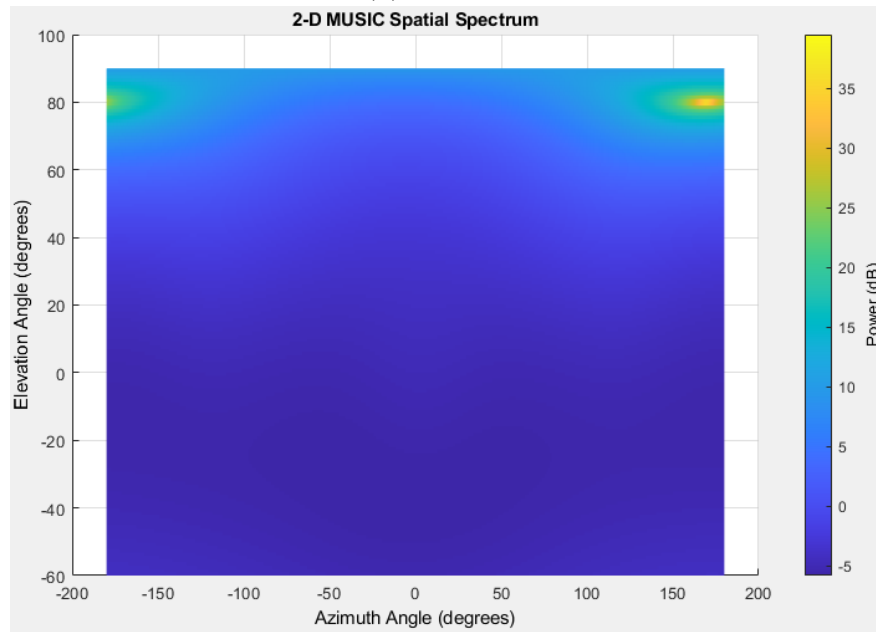
(a) *Broadside.*(b) *Endfire.*

Figura 3.8: Resolución de MUSIC en distintos ángulos para arreglo tetraédrico de 4 elementos.

Capítulo 4

Diseño e implementación

Al igual que en las simulaciones, la implementación en hardware del sistema se realizó de forma progresiva: se comenzó con la solución más simple (de dos micrófonos, estimación en una dimensión, algoritmo sencillo, un solo dispositivo para captura y procesamiento) y luego con la más compleja (cuatro micrófonos, estimación en dos dimensiones, algoritmo más sofisticado, un dispositivo para captura y otro para procesamiento).

Se describirán los diseños finales de ambos sistemas, el funcionamiento paso a paso de cada uno, el por qué de distintas decisiones de diseño, las limitaciones que presentan, cómo se podrían mejorar y, finalmente, se resumen y comparan ambos sistemas.

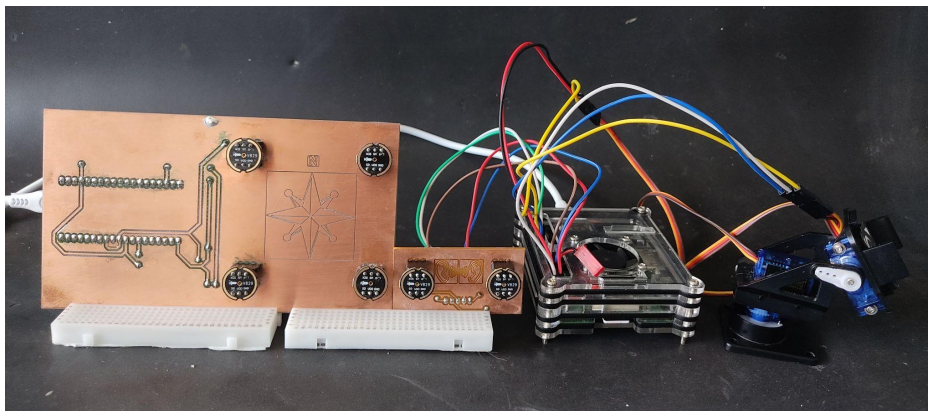


Figura 4.1: La implementación final. De izquierda a derecha: el arreglo de 4 micrófonos, el arreglo de 2 micrófonos, la Raspberry Pi y los servomotores.

4.1. Arreglo de dos micrófonos con GCC-PHAT

El primer diseño implementado consiste de un arreglo lineal de dos micrófonos a una distancia de 3,5 cm que estima la dirección de arribo en una única dimensión (en un rango de 180°) mediante el algoritmo GCC-PHAT. La captura de audio se realiza con dos INMP441 mediante un único bus I2S directamente

con la Raspberry Pi. Además de estimar DOA en tiempo real y mostrarla en una terminal, utiliza un servomotor para apuntar a la dirección estimada y registra los 'eventos' acústicos (aquellos que superen un umbral de energía) en un archivo .csv.

La frecuencia máxima no ambigua, con la separación de 3,5 cm entre micrófonos, es de 4900 Hz; como este sistema no está pensado para ninguna fuente particular (como el caso del dron en el otro sistema, que está entre 200 y 2400 Hz), y como la ponderación PHAT le da el mismo peso a cada bin de frecuencia analizada, nos interesa quitar todas las bandas que no puedan contener una señal útil. Para esto se aplica un filtro pasa banda con frecuencia de corte inferior de 200 Hz y frecuencia de corte superior de 4400 Hz (90 % de la frecuencia máxima no ambigua)

Cuenta con dos modos de uso que modifican el funcionamiento del servomotor: el modo “seguimiento” realiza un seguimiento continuo de toda señal que no considere silencio, mientras que el modo “evento” realiza un solo movimiento hacia la dirección en la que detecta un evento y se queda ahí hasta que se detecta uno nuevo. Este sistema no se ideó específicamente para localizar drones sino para cualquier sonido. Para probarlo se utilizaron aplausos y la grabación de un motor de dron desde los parlantes de un teléfono para los modos “evento” y “seguimiento”, respectivamente.

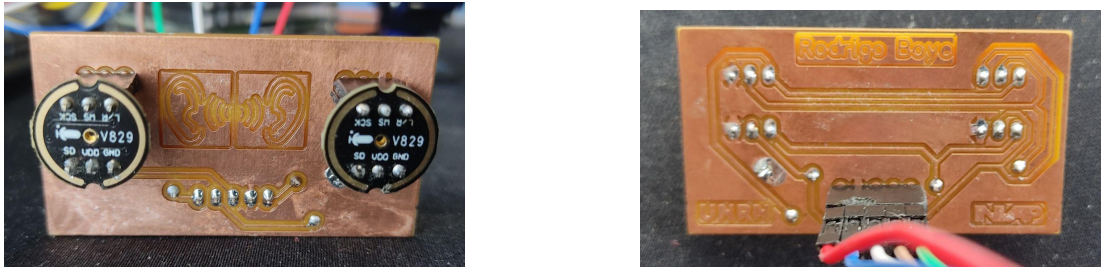


Figura 4.2: Vista frontal y trasera del sistema de dos micrófonos.

4.1.1. Cadena de procesamiento

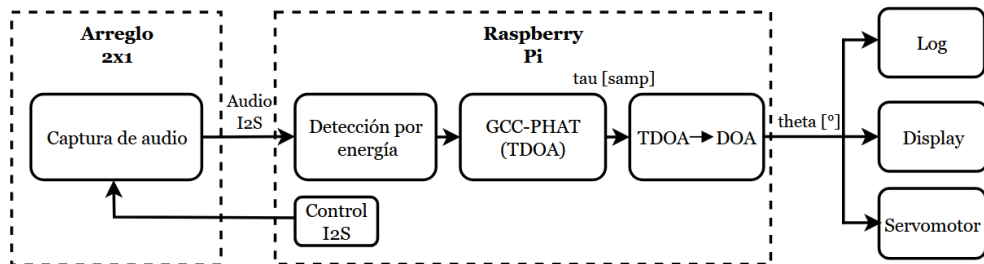


Figura 4.3: Cadena de procesamiento del sistema de 2 micrófonos.

- **Captura de audio:** los micrófonos del arreglo graban una señal estéreo I2S (canal izquierdo y derecho, un único bus) controlados por las señales CLK y WS provenientes de la Raspberry Pi. El control y conexión directas permiten grabar en 24 bits con una frecuencia de muestreo de 48 kHz; esto da mayor precisión al calcular el TDOA ya que se calcula en muestras de diferencia, más allá de las frecuencias de audio que nos interesan y el aliasing espacial.
- **Detección por energía:** calcula la energía media del bloque de procesamiento, si la energía es menor a la variable `umbral_ruido` se considera silencio/ruido y se descarta. El tamaño del bloque es configurable, puede ser menor (por ejemplo 128 muestras) para estimar con más rapidez o mayor (2048 muestras) para ser más preciso
- **Estimación de TDOA con GCC-PHAT:** se aplica el algoritmo desarrollado en la Sección 2.4.2, aplicando un filtro pasabanda de 200-4400 Hz entre la FFT y el espectro cruzado de potencia; como resultado se obtiene un retardo τ en muestras. El máximo de correlación se busca primero como un índice entero de muestras y luego se refina con interpolación parabólica para conseguir resolución sub-muestra; esto es necesario porque, para 3,5 cm de separación entre sensores y $f_s = 48 \text{ kHz}$, los 180° de rango abarcan aproximadamente 10 valores discretos.
- **Conversión de TDOA a DOA:** conociendo el retardo, ahora convertido a segundos, y la distancia entre micrófonos, se calcula simplemente como

$$\theta = \cos^{-1}(\tau \cdot v/d) \quad (4.1)$$

- **Registro de eventos:** si la energía supera la variable `umbral_evento` (proporcional al umbral de ruido) se abre un evento; mientras este continúe se acumulan ángulo, confianza de la estimación y energía medida, y cuando termina se registra en un archivo `.csv` junto con el tiempo de inicio y fin.
- **Display:** se imprime una barra ASCII en la terminal de la Raspberry Pi con la posición estimada en tiempo real.
- **Servomotor:** opcional; en caso de estar habilitado controla un servomotor para que “apunte” a la dirección estimada. Controlarlo con la estimación cruda causaría que tiemble mucho y se comporte de manera errática, por lo que se incorporan los siguientes mecanismos:
 - *Promediado:* acumula varias estimaciones y mueve el servo al promedio, filtrando el jitter en la estimación.

- *Zona muerta*: no se mueve si la nueva posición está muy cerca de la actual, esto evita oscilaciones irrelevantes.
- *Apagado del PWM (detach)*: deja de enviar pulsos cuando llega a la posición deseada ya que el PWM puede causar jitter por software (el ancho de los pulsos generados no es exactamente igual); el motor mantiene la posición por fricción interna.

4.1.2. Decisiones de diseño, limitaciones y posibles mejoras

Este primer prototipo se diseñó con el objetivo de ser una base, un punto de partida desde el cual comenzar a experimentar con los sistemas de localización acústica, y es por eso que se priorizó la sencillez por sobre la precisión o la aplicabilidad.

Buscando esta sencillez, se toma la decisión de utilizar el arreglo más simple posible (dos micrófonos) y uno de los algoritmos más simples (GCC-PHAT), asumiendo a priori que serviría para estimar una sola dimensión con una precisión limitada. El detector también es simple: se limita a medir la energía recibida en los micrófonos, sin distinguir el tipo de sonido que se recibe (frecuencia, patrones, firmas acústicas, etc.); este detector se mantiene en el segundo diseño y es una de las posibles mejoras que se podrían realizar.

Con la distancia de 3,5 cm entre micrófonos y 48 kHz de frecuencia de muestreo, que permitiría grabar sonido de hasta 24 kHz (aliasing temporal), el TDOA máximo que puede existir es de 4,9 muestras ($= f_s d/v$), lo que limita la precisión con la que puede calcular la DOA, sobre todo en los extremos. Además, la separación limita la frecuencia máxima de estimación por aliasing espacial a 4900 Hz ($= v/2d$). Si se aumenta la distancia entre micrófonos se aumenta la precisión, pero se reduce el rango de frecuencias con las cuales se puede trabajar. Para este prototipo, 3,5 cm de separación se consideró un buen compromiso, pero en caso de necesitar más precisión o mayores frecuencias se puede modificar fácilmente.

4.2. Arreglo de cuatro micrófonos con MUSIC

El segundo y último sistema que se implementó consiste en un arreglo cuadrado (o circular) de cuatro micrófonos con 5 cm de lado que estima la dirección de arribo de la señal en dos dimensiones, acimut y elevación, mediante el algoritmo MUSIC, aunque también incluye el algoritmo SRP-PHAT como alternativa. En

este caso, el control de los micrófonos es realizado por un ESP32, que se limita a grabar los cuatro canales y transmitirlos por USB a la Raspberry Pi, que sigue estimando la DOA, imprimiéndola en la terminal, controlando los dos servomotores (uno para cada dimensión) y registrando los eventos. Se mantuvo la separación de modos “evento” y “seguimiento” bajo los mismos principios. Este sistema sí se pensó con el objetivo de localizar drones, por lo que se tomaron decisiones de diseño que apuntan a mejorar el desempeño en un caso real.

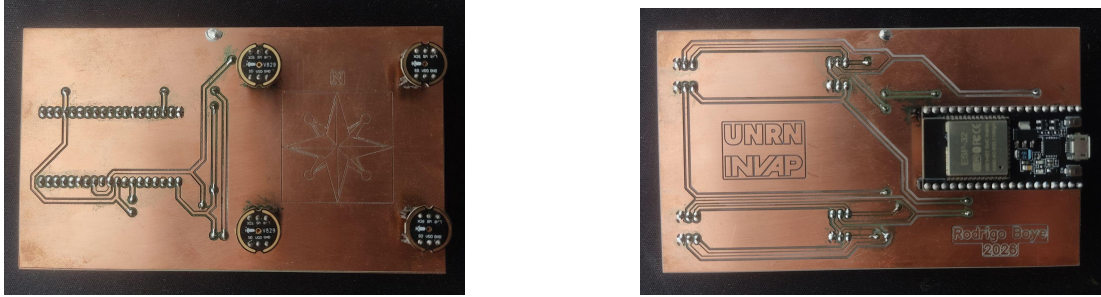


Figura 4.4: Vista frontal y trasera del segundo diseño implementado.

4.2.1. Cadena de procesamiento

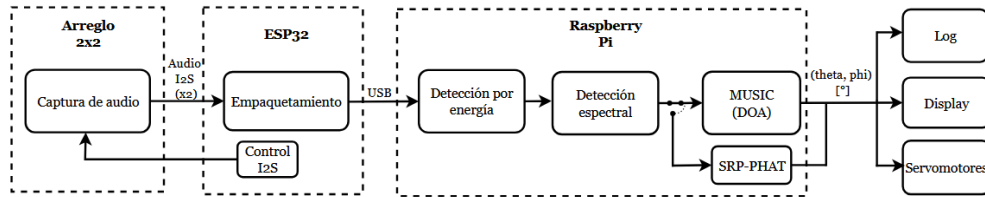


Figura 4.5: Cadena de procesamiento del sistema de 4 micrófonos.

- **Captura I2S:** dos buses I2S ($\text{bus0}=[\text{mic0},1]$ y $\text{bus1}=[\text{mic2},3]$) capturados simultáneamente por el ESP32. La captura está configurada con frecuencia de muestreo de 11025 Hz a 16 bits y los dos buses se controlan en modo master + slave con reloj compartido.
- **Empaquetamiento:** para transmitir por USB serial desde el ESP32 a la RPi, las señales se empaquetan en hops con byte de sincronismo (0xAA), contador de dos bytes (para detectar paquetes perdidos), las muestras (256 muestras, 4 canales de 16 bits) y otro byte de sincronismo (0x55).

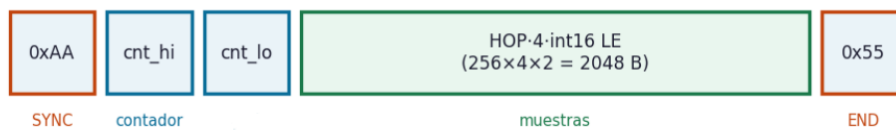


Figura 4.6: Estructura del paquete serial por hop (de 256 muestras, modificable).

- **Detección por energía:** luego de leer la entrada USB, se mide la energía de la señal. Tiene una complejidad mayor al sistema anterior, considera un piso de ruido que se puede medir al comienzo de la grabación o hardcodear, utiliza histéresis (el umbral para comenzar el evento es mayor que el umbral para finalizarlo) y funciona como una máquina de cuatro estados: IDLE (en silencio, esperando una señal), ONSET (cuando se supera el umbral de evento espera a confirmarlo durante varios frames, durante los cuales se llena la matriz de covarianza, ec. 2.25), ACTIVE (evento confirmado, en funcionamiento) y COOLDOWN (espera antes de volver a confirmar un evento para no volver a detectar el que está acabando). La energía se mide dentro del rango de frecuencias de MUSIC para evitar que se active con infrasonido que no puede ser la señal útil.

- **Detección espectral:** el detector por energía se dispara con cualquier sonido fuerte, pero el objetivo del sistema es rastrear específicamente drones, por esto se implementa una segunda etapa de detección que solamente se activa con ellos. Se aprovecha la firma acústica distintiva de los rotores del dron: cada uno genera un tono a la Frecuencia de Paso de Pala (BPF, igual al número de palas $\times$ RPM/60) que suele estar entre 80-300 Hz, y una serie de armónicos de esta frecuencia; el detector busca reconocer este “peine”.

Para estimar la BPF se utiliza el Espectro de Productos Armónicos (HPS): se multiplica el espectro de magnitudes por versiones decimadas de sí mismo (por una comprimida a la mitad, a un tercio, y así sucesivamente), de esta forma el primer armónico ($2 \times f_{BPF}$) caerá exactamente en f_{BPF} en la versión comprimida a la mitad, el segundo armónico caerá en f_{BPF} en la versión comprimida a un tercio, y así de manera sucesiva; si la señal es efectivamente armónica, todos los armónicos reforzarán f_{BPF} mientras que las demás frecuencias serán suprimidas al multiplicarse por ruido (Fig. 4.7).

$$HPS(f) = \prod_{r=1}^R |X(r \times f)|, \quad BPF = \operatorname{argmax} HPS(f) \quad (4.2)$$

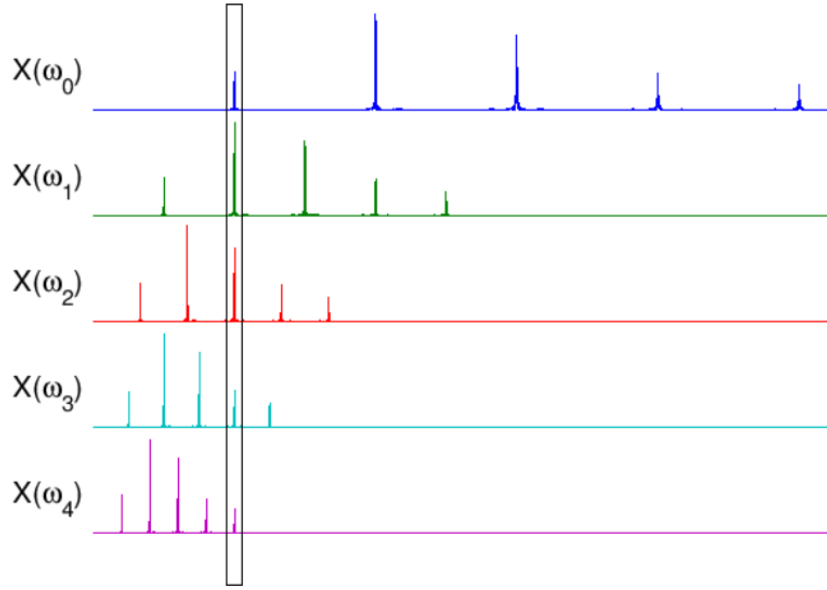


Figura 4.7: Estimación de BPF mediante Espectro de Productos Armónicos (fuente: musicweb.ucsd).

Luego se mide el SNR de los posibles armónicos (múltiplos de la BPF estimada) contra el máximo entre la mediana local y el piso global. Para confirmar la detección se necesita que haya al menos tres armónicos con SNR suficiente, al menos uno dentro de la banda de MUSIC y que una fracción de la energía del peine supere un mínimo. Este detector se puede desactivar para probar el sistema con otros sonidos. Se puede ver la diferencia entre la señal de un dron real (Fig. 4.8) y la de una grabación reproducida desde un parlante (Fig. 4.9).

- **MUSIC:** se aplica MUSIC de banda ancha incoherente y escaneado en espacio-u, algoritmo desarrollado en la Sección 2.4.4; como resultado se obtienen los ángulos de acimut φ y elevación θ .
- **SRP-PHAT:** se incluye el algoritmo SRP-PHAT (Sección 2.4.3) como alternativa para aquellos casos en los que MUSIC falla; se comparan con profundidad en la Sección 4.2.3.
- **Registro, display y servomotores:** mismo funcionamiento pero incluyendo el segundo ángulo (otra columna en el log, otro eje en la terminal y otro servomotor).

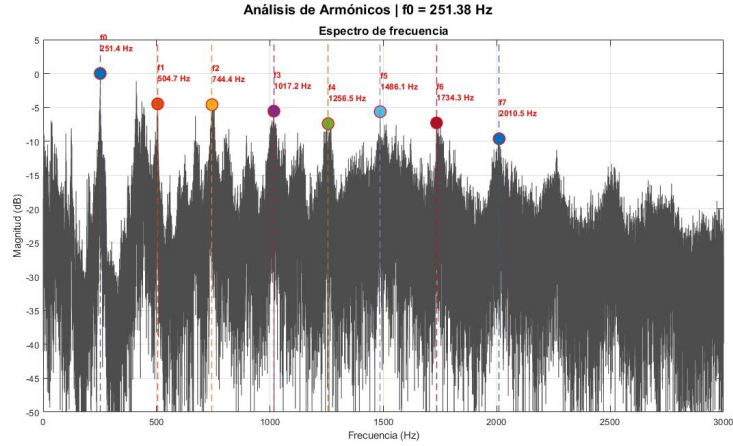


Figura 4.8: Espectro de frecuencia de un dron real. Se aprecia claramente el BPF en 250 Hz y los armónicos de mayor magnitud dentro del rango de frecuencias de MUSIC (200–2400 Hz).

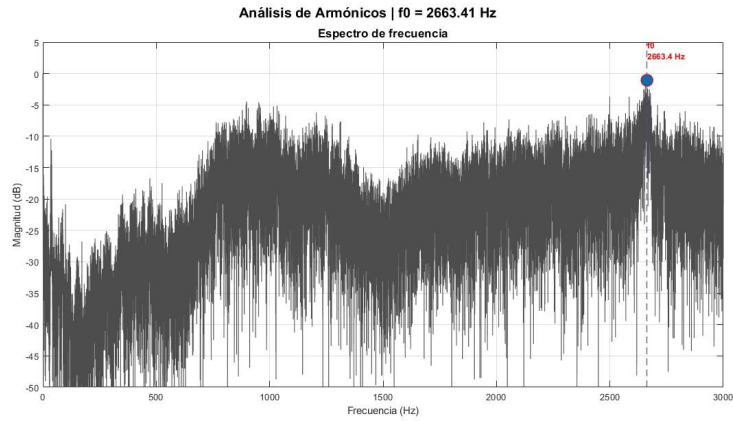


Figura 4.9: Espectro de la reproducción de la grabación de un dron desde el parlante de un celular; se pierden los bajos y el comportamiento armónico, no activa el detector espectral.

4.2.2. Decisiones de diseño, limitaciones y posibles mejoras

Se eligió un arreglo planar por sobre uno volumétrico por la simplicidad de construcción: se puede imprimir sobre una única plaqueta, una estructura sólida donde la posición de los micrófonos es precisa y está fija, las líneas I2S son cortas y de longitud comparable por lo que la sincronización es mejor. Un arreglo volumétrico, por su parte, requiere múltiples planos, más difíciles de diseñar y construir correctamente. Estas razones pesaron más que las ventajas proporcionadas por un arreglo volumétrico (resolución más uniforme y cobertura angular de 360°) observada en las simulaciones del Capítulo 3.

La frecuencia máxima por aliasing espacial la define la distancia más grande entre micrófonos; para un cuadrado de 5 cm de lado, la diagonal es de 7,07 cm y limita la frecuencia a 2425 Hz. Se configura la frecuencia máxima utilizada para el algoritmo en 2400 Hz, lo que contiene la mayor parte de la señal útil. También debido al tamaño del arreglo, a la frecuencia de la señal, a la frecuencia de muestreo y al escaneado en espacio-u, se determina un rango de $\pm 75^\circ$ en acimut; el sistema no tiene resolución útil en ángulos más abiertos.

Para aprovechar la resolución máxima en el centro del arreglo, el rango en elevación también es simétrico respecto al eje, más precisamente de $\pm 35^\circ$. Como se considera que los objetivos estarán más arriba que el arreglo y no tiene sentido escanear elevaciones negativas, se incluyó la posibilidad de inclinarlo para desplazar el rango: simplemente se configura el ángulo de inclinación física o *tilt* del arreglo y se suma a la estimación.

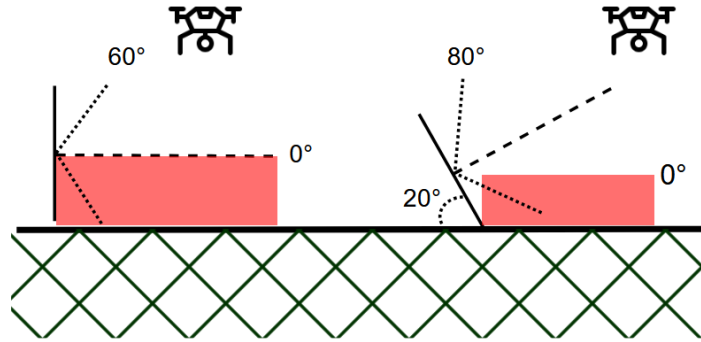


Figura 4.10: Traslado de rango con inclinación del arreglo.

Usar el puerto serie USB para la transmisión entre ESP32 y Raspberry Pi causa un cuello de botella, limita el datarate a 92 KB/s (el baudrate máximo del ESP32 utilizado es de 921600). Para que los datos se encuentren bajo ese límite, se eligió grabar a 11 kHz y cuantificar con 16 bits ($4 \times 2 \times 11025 = 88,2$ KB/s). El compromiso entre frecuencia de muestreo y resolución en bits se puede resumir así: en el caso específico de MUSIC, una mayor frecuencia de muestreo implica más snapshots en el mismo tiempo para estimar la matriz de covarianza (una mejor estimación) y una mayor resolución de fase, mientras que una mayor resolución en bits implica menos ruido de cuantización (mejor SNR). Si, en cambio, quisiéramos favorecer el rendimiento del motor SRP-PHAT o cualquier algoritmo de TDOA, se debería aumentar la frecuencia de muestreo (8 bits x 22 kHz) ya que aquí la resolución de fase es más importante.

En el pseudoespectro de MUSIC (ec. 2.27) se multiplica $\mathbf{a}^H \mathbf{V}_n \mathbf{V}_n^H \mathbf{a}$, de esta forma cada punto en la grilla y cada bin exigen un producto matriz vector de $M \times M$, lo que dominará el tiempo de ejecución en un dispositivo de poca memo-

ria como la Raspberry Pi. Como los autovectores forman una base ortonormal completa

$$\mathbf{V}_s \mathbf{V}_s^H + \mathbf{V}_n \mathbf{V}_n^H = \mathbf{I} \quad (4.3)$$

por lo tanto se puede reemplazar

$$\mathbf{a}^H \mathbf{V}_n \mathbf{V}_n^H \mathbf{a} = \mathbf{a}^H \mathbf{a} - \|\mathbf{V}_s^H \mathbf{a}\|^2 \quad (4.4)$$

y, dado que el vector de dirección tiene módulo unitario en cada componente (contiene solo información de fase, ec. 2.28), $\mathbf{a}^H \mathbf{a} = \sum |\mathbf{a}[m]|^2 = 1 + 1 + \dots = M$ exactamente. El denominador puede entonces calcularse como

$$\mathbf{a}^H \mathbf{V}_n \mathbf{V}_n^H \mathbf{a} = M - \|\mathbf{V}_s^H \mathbf{a}\|^2 \quad (4.5)$$

Asumiendo una sola fuente, el subespacio de señal es un único vector: la operación se reduce a un producto interno de M valores en lugar de un producto matriz-vector, reduciendo la memoria utilizada y favoreciendo el rendimiento. Matemáticamente, la identidad es exacta, por lo que el pseudoespectro producido es igual al obtenido mediante el subespacio de ruido. Al ser estimados con muestras finitas, los subespacios no coinciden exactamente con los verdaderos: una parte de la energía de la señal se filtra hacia el subespacio de ruido en un fenómeno conocido como *subspace leakage*, por esta razón el denominador nunca llega a ser cero ni el pico del pseudoespectro, infinito (Friedlander, 2009; Chen et al., 2010).

La variante de MUSIC utilizada separa el espectro de banda ancha en bins para aplicar el algoritmo, pero lo hace de forma continua, desde la frecuencia mínima hasta la máxima. Existe una variante llamada MUSIC con Extracción de Sub-banda (MUSIC-SE), utilizada en (Zhu et al., 2021), que aprovecha el espectro irregular de determinadas señales y hace una extracción o selección de frecuencias: utiliza solamente los bins de frecuencia con SNR más alto, en nuestro caso los armónicos, para reducir el costo computacional sin degradar los resultados. MUSIC-SE se consideró y evaluó experimentalmente antes de descartarlo: reducir la cantidad de bins aumenta la varianza de la estimación y los errores, los desechados siguen conteniendo señal, sólo con menor SNR, y la mejora computacional no es tan significativa en nuestro caso. Se deja mencionado sin embargo como posible mejora en caso de querer implementar el sistema con más datos (arreglos más grandes, cuantización de más bits, mayor frecuencia de muestreo) y/o hardware de menos potencia que requiera estrictamente mayor eficiencia.

Para asegurar el sincronismo entre buses se utiliza una configuración master/slave: el bus 0 es el “maestro” que genera señales CLK y WS mientras el bus 1 es el “esclavo” que sigue las señales del maestro, así nos aseguramos que no haya desviaciones entre dos relojes distintos.

Al ejecutar el programa se puede elegir entre los modos “evento” y “seguimiento”. El modo “seguimiento” es el default, utiliza MUSIC y realiza una acumulación de datos para reducir el error sobre fuentes continuas; el modo “evento”, por su parte, utiliza de forma predeterminada SRP-PHAT y funciona de forma inmediata, cuando se supera el umbral de energía no espera a confirmar el evento, pasa directamente al estado ACTIVE.

4.2.3. Comparación MUSIC y SRP-PHAT

Si bien el sistema implementa los dos algoritmos de forma intercambiable, se elige a MUSIC como el algoritmo por defecto para el seguimiento de drones y SRP-PHAT como el predeterminado para los eventos rápidos, transitorios. Las razones para esta decisión son las siguientes:

1. El blanco de interés principal para el sistema, un dron en vuelo, es una fuente sostenida y de estructura de frecuencia definida: su energía se concentra en la frecuencia de paso de palas (BPF) y sus armónicos. Este tipo de fuente es donde MUSIC puede definir el subespacio de señal de mejor forma y conseguir un pico más angosto, es decir, donde tendrá mejor resolución angular.
2. El escenario pensado para el sistema es al aire libre, donde, si bien puede haber ruido causado por el viento u otros fenómenos, las posibilidades de encontrar ruido correlacionado por reverberación son mucho menores que en un ambiente cerrado. MUSIC presenta dificultades para separar el ruido correlacionado de la señal ya que se basa en descomponer la matriz de covarianza, por lo que este tipo de ruido debe ser mínimo. En caso contrario, puede ser preferible SRP-PHAT u otro algoritmo.
3. Las razones anteriores contemplan la detección de drones como uso principal del sistema, pero un objetivo del proyecto es que también se pueda localizar la detonación de artillería o eventos similares. En estos casos las razones anteriores se invierten: no hay una estructura armónica de la señal sino un espectro de banda ancha y plana, y la fuente no es sostenida sino impulsiva y corta, se extingue antes de poder condicionar correctamente la matriz de covarianza. Aquí es donde SRP-PHAT ofrece mejor rendimiento que

MUSIC, por eso al ejecutar el modo “evento” del programa principal se utiliza SRP-PHAT mientras que el modo “seguimiento” aplica MUSIC.

Computacionalmente, SRP-PHAT (GCC entre las seis combinaciones de cuatro micrófonos, fundamentalmente FFTs) es mucho más eficiente que MUSIC (multiplicar matrices y escanear los pseudoespectros bidimensionales de todos los bloques). Sin embargo, con la configuración de `hop_size= 256` y frecuencia de muestreo de 11025 Hz, el tiempo entre cada hop que ingresa es de 23 ms, y con el procesamiento de la Raspberry Pi ambos algoritmos pueden realizarse, por lo que no condiciona la decisión. Se menciona de todas formas para tener en cuenta en situaciones con más elementos o menos capacidad de procesamiento. Al elegir MUSIC como motor principal, la configuración del sistema se ajusta a lo que favorezca a este algoritmo, en detrimento de SRP-PHAT; en caso de priorizar este otro, convendría cambiar el formato de los datos a 8 bits x 22 kHz, tanto en el ESP32 como en la Raspberry Pi.

4.3. Resumen y comparación

Podemos comparar las características de los dos sistemas implementados de la siguiente forma:

Tabla 4.1: Comparación entre sistemas.

Característica	2 micrófonos	4 micrófonos
Objetivo	Pruebas, familiarización, localizar cualquier sonido cercano	Detección y localización de drones
Salida	1 ángulo (acimut o elevación)	2 ángulos (acimut y elevación)
Entrada	1 bus I2S – 2 canales – 48 kHz 24 bits	2 buses I2S – 4 canales – 11 kHz 16 bits
Geometría	Lineal, separación de 3,5 cm	Cuadrada, 5 cm de lado
Algoritmo	GCC-PHAT	MUSIC (principal) y SRP-PHAT (alternativo)
Hardware	2 micrófonos MEMS INMP441 y 1 Raspberry Pi 3A+	4 micrófonos MEMS INMP441, 1 ESP32 y 1 Raspberry Pi 3A+
Detector	Por energía	Por energía y luego espectral
Adicionales	Registro de eventos, display en terminal y control de 1 servomotor	Registro de eventos, display en terminal y control de 2 servomotores
Ventajas	Simpleza, rapidez, bajo costo y consumo	Mayor resolución, mejor rechazo de ruido no correlacionado, específico para drones, posibilidad de cambiar de algoritmo
Debilidades	Un solo ángulo, menor resolución, sin discriminación para dron, más sensible al ruido	Mayor complejidad y puntos posibles de falla, limitado por enlace USB, sensible a reverberación

Capítulo 5

Pruebas y resultados

Se realizaron pruebas para evaluar el desempeño de ambos sistemas en distintas condiciones.

5.1. Sistema de dos micrófonos

Al ser un prototipo de aprendizaje, sin el objetivo de localizar drones en situaciones reales, se testeó de forma cualitativa y no cuantitativa (no se realizaron mediciones precisas) y solamente en ambientes cerrados, pero con distintos niveles de ruido ambiental y reverberación.

En primera instancia, mientras se desarrollaba y fabricaba, fue probado en una habitación pequeña, reverberante pero en completo silencio. Aquí es donde se corrigieron y ajustaron distintas partes del sistema: se calibró el servomotor para que reproduzca el ángulo estimado de forma fluida y precisa, se probaron distintas separaciones entre micrófonos, se ajustaron los niveles de los umbrales de ruido y evento para que se dispare correctamente, se buscó un punto de equilibrio en el tamaño del bloque de procesamiento (la cantidad de muestras por cada estimación) para lograr un funcionamiento rápido pero preciso, entre otros ajustes. Una vez realizado esto se comprobó el correcto funcionamiento del sistema tanto en modo “evento”, probado con aplausos, como en modo “seguimiento”, probado reproduciendo el ruido de drones desde un parlante.

Luego, el sistema fue probado y exhibido en Invap, en el contexto de una muestra por sus 50 años. En este caso el lugar de prueba fue una sala mucho más grande en la que no había reverberación pero sí mucho ruido ambiental debido al público presente. Aquí también, luego de ajustar manualmente los umbrales de sonido, se obtuvieron estimaciones correctas.

Se observó que el sistema cumple con las características planeadas al implementar GCC-PHAT: ofrece una precisión suficiente para su aplicación (el servomotor apunta adecuadamente a la fuente de sonido), funcionamiento rápido y

robustez frente a la reverberación, mientras que la señal que se quiere estimar sea más fuerte que el ruido ambiental.

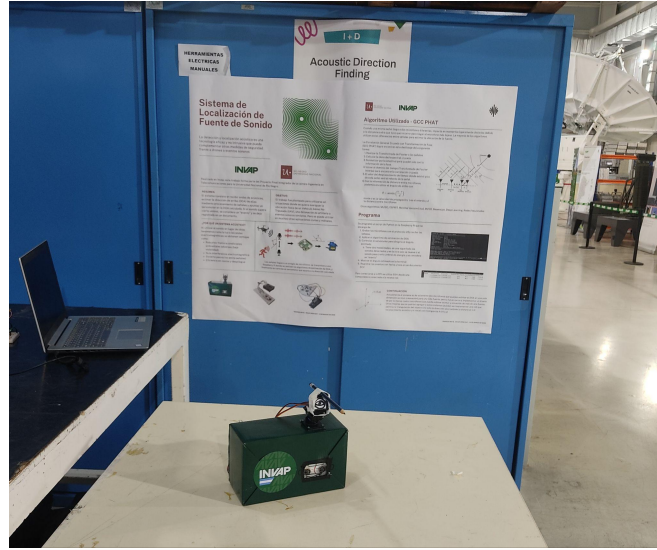


Figura 5.1: Sistema de dos micrófonos en muestra de Invap.

5.2. Sistema de cuatro micrófonos

En el segundo sistema, en cambio, se requieren pruebas más rigurosas para asegurar el correcto funcionamiento en las situaciones de aplicación pensadas.

Primero se probaron los algoritmos de estimación de DOA sin el detector espectral con aplausos y grabaciones de dron dentro de una habitación cerrada y pequeña. Aquí se pudo comprobar que MUSIC se degrada mucho con la reverberación, ya que al probarlo en modo “evento” con aplausos no puede estimar correctamente la dirección, mientras que SRP-PHAT, un algoritmo menos avanzado, sí puede. El rendimiento en modo “seguimiento” con sonido continuo fue más parejo ya que le da tiempo a MUSIC a realizar una mejor estimación del subespacio de señal, y en sonidos más continuos, menos impulsivos, la reverberación pierde un poco de peso frente a la señal principal. Probando el sistema con un dron real (DJI Mini 4 Pro, Apéndice B.1.4) dentro de una habitación amplia se verificaron el funcionamiento del detector espectral y el correcto seguimiento con el algoritmo MUSIC. Para realizar la Tabla 5.1 se centró la señal en broadside de forma que la estimación sea $[0,0]$, se giró el arreglo a determinados ángulos con la guía de un transportador y se anotó el valor de la estimación correspondiente. Este proceso se repitió cinco veces, pero las mediciones no son extremadamente precisas, y al centrar primero la estimación, en vez del ángulo físico real, en cero, no se puede medir cualquier sesgo que pueda tener el sistema, por lo que la siguiente tabla ofrece una caracterización general más que datos exactos:

Tabla 5.1: Estimación con MUSIC en sistema de cuatro micrófonos, ambiente cerrado, distancia corta.

Ángulo real [ac, el]	Ángulos estimados (media $\pm$ desvío, aproximado) [ac, el]	RMSE
[0, 20]	[0.0 $\pm$ 1, 20.2 $\pm$ 1.5]	1.8
[20, 20]	[18.5 $\pm$ 2.5, 20.3 $\pm$ 1.5]	3.3
[40, 20]	[38.5 $\pm$ 4, 20.8 $\pm$ 1.8]	4.7
[60, 20]	[57.0 $\pm$ 5.5, 20.5 $\pm$ 1.5]	6.5
[0, 40]	[0.2 $\pm$ 1, 36.5 $\pm$ 6.0]	7.0

Finalmente, se realizó una prueba final en un escenario similar a un caso real de aplicación: la detección de un dron en campo abierto. Las condiciones específicas del clima fueron temperatura de 10°C , por lo que se ajustó la velocidad de propagación a 337 m/s, y viento de 12–18 km/h. El dron con el que se contó para las pruebas fue el DJI Mini 4 Pro (Apéndice B.1.4), un modelo especialmente silencioso (prácticamente imperceptible a 50 metros), lo que dificulta la evaluación del sistema al mismo tiempo que evidencia las limitaciones intrínsecas de la detección acústica: se requiere una sensibilidad muy alta para poder percibir ruidos de tan bajo nivel, sobre todo si el ambiente es ruidoso.

En la prueba se comprobó el problema de sensibilidad. La detección y localización funcionaron correctamente a una distancia de aproximadamente 2 metros y de 5 metros: sin realizar mediciones precisas se observó cómo el servomotor apuntaba al dron, siguiéndolo a medida que se movía horizontal y verticalmente; se considera que funciona con tendencia similar a la Tabla 5.1, pero con errores mayores debido al peor SNR. A distancia de 10 metros la detección de la señal era esporádica, no siempre pasaba ambos umbrales, y la localización, cuando efectivamente se detectaba la señal, era errática, fallaba. A distancias mayores a 10 metros ya ni siquiera se detectaba la señal.



Figura 5.2: Prueba del sistema de cuatro micrófonos con un dron en campo abierto.

Para presentar una caracterización del funcionamiento del sistema se simuló sobre el *pipeline* MUSIC Incoherente real múltiples señales armónicas con DOAs conocidas y distintos SNR. La simulación asume el caso más favorable: ruido blanco e independiente entre canales y una sola fuente, por lo que no predice el comportamiento exacto del diseño en un caso real, pero es útil para observar la degradación por bajo SNR (Fig. 5.3) y ángulos de arribo cercanos a endfire (comienza a tener mayor error cuando se acerca al límite de 70° y, fuera de los límites del gráfico, el error sigue aumentando hasta los 90° , Fig. 5.4).

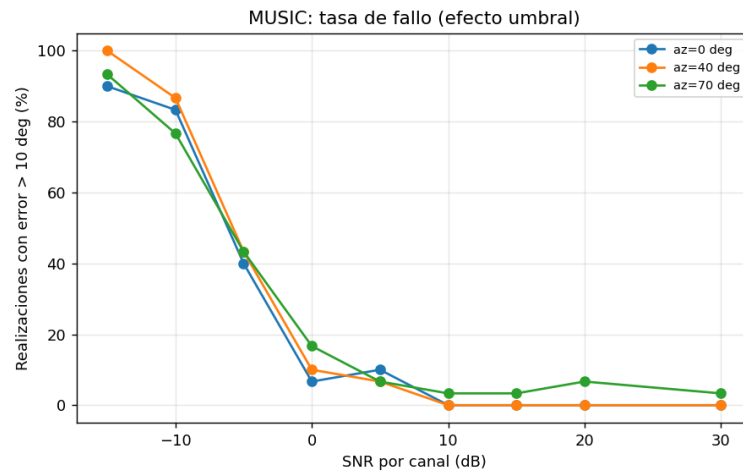


Figura 5.3: Simulación: tasa de error mayor a 10° en MUSIC según SNR.

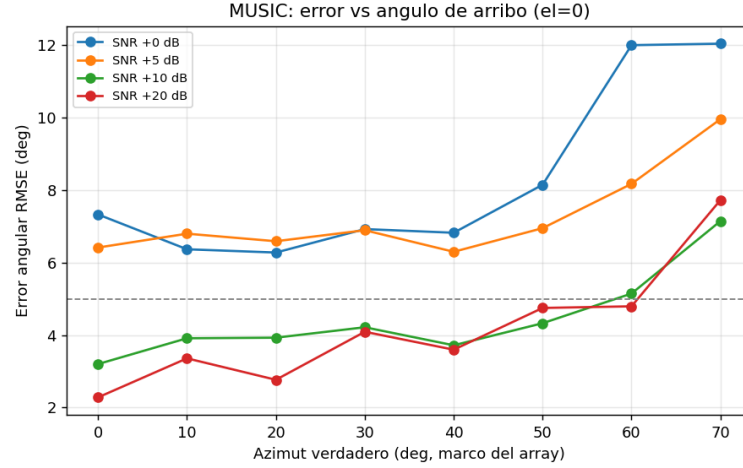


Figura 5.4: Simulación: raíz de error cuadrático medio (RMSE) en MUSIC según SNR y ángulo de arribo.

Para poder ejecutarse en tiempo real, con una tasa de 11025 Hz y bloques de 256 muestras, el presupuesto disponible es de 23,2 ms por trama, dentro los cuales deben entrar la lectura de datos por puerto serial, el detector espectral, el algoritmo (MUSIC o SRP-PHAT) y el control de los servomotores. El proceso más exigente es MUSIC, principalmente el escaneo de la grilla: con un *script* de medición se determinó que todo el lazo con MUSIC, con la configuración actual, tarda aproximadamente 20 ms. Cuando por algún motivo se pasa el límite se comienzan a descartar tramas, invalidando la estimación.

Capítulo 6

Conclusiones y posibles líneas futuras

6.1. Conclusiones generales

Este trabajo documentó el diseño, la fundamentación teórica y la implementación de dos sistemas de localización de fuente de sonido que estiman en tiempo real la dirección de arribo de una señal acústica.

Desde el marco teórico se destaca la idea que determina a este tipo de sistema: la información sobre la dirección de arribo de la fuente reside en las diferencias, de tiempo/fase, entre las señales captadas por cada sensor. De eso se desprenden los requisitos del diseño (sincronización entre canales, conocimiento preciso de la geometría del arreglo), los límites (frecuencia máxima por aliasing espacial, precisión máxima por apertura) y los fundamentos de todos los algoritmos, que utilizan distintas técnicas para estimar estas diferencias.

El sistema “final”, conformado por un arreglo cuadrado de cuatro micrófonos, utiliza el algoritmo MUSIC Incoherente sobre hardware embebido para localizar específicamente drones (o SRP-PHAT para eventos impulsivos, similares a una detonación de artillería), registrar el evento y apuntar dos servomotores hacia el objetivo. El control de la grabación de dos interfaces I2S es realizado por un ESP32, que transmite las señales por USB a una Raspberry Pi que se encarga de realizar la estimación. Antes de ejecutar el algoritmo se deben superar dos umbrales detectores: el primero es de energía y se activa cuando se detecta cualquier sonido, el segundo es espectral y se activa cuando el sonido detectado presenta el comportamiento armónico típico de los drones. El algoritmo del sistema es MUSIC, un algoritmo de subespacio que se eligió por su alta resolución, su buen rendimiento frente a ruido no correlacionado y su independencia de la frecuencia de muestreo (limitada por la transmisión serial). Se adoptaron criterios de robustez y simplicidad para el diseño: módulos o secciones independientes y claras,

testeadas por separado y en conjunto.

El sistema no busca superar en rendimiento a sistemas del estado del arte ya que existen sistemas mucho más avanzados, con redes de decenas de micrófonos y algoritmos muy complejos, que ofrecen precisión extremadamente alta, además de prestaciones adicionales. Lo que busca, en cambio, es implementar un sistema completo, funcional, de costo mínimo y que sirva de base conceptual para futuras mejoras.

Habiendo realizado múltiples pruebas tanto del sistema de dos micrófonos como del sistema de cuatro micrófonos, tanto en ambientes cerrados y controlados como en ambientes abiertos, ruidosos y/o con viento, podemos decir que los sistemas implementados realizan correctamente la detección y localización bajo condiciones favorables. La caracterización por simulación sobre el pipeline real sitúa en torno a 10 dB de SNR el punto donde el error comienza a crecer con rapidez. Se verificó el correcto funcionamiento de cada módulo o parte del diseño: la toma de datos, la detección por energía y firma espectral, los algoritmos de estimación de DOA y las funciones adicionales. Se comprobó también que distintos algoritmos pueden ofrecer mejores o peores resultados dependiendo de las características del escenario y la señal con las que se esté trabajando, por lo que es fundamental analizar cada situación particular antes de decidir por un diseño específico.

Se puede concluir, entonces, que se cumplen en gran medida los objetivos planteados al comienzo del trabajo: se implementó un sistema que detecta y estima la dirección de arribo de la señal acústica de un dron en tiempo real, utilizando hardware de bajo costo, y se cumplen también, por lo tanto, los objetivos específicos.

Sin embargo, el sistema aún no está en condiciones de aplicarse en una situación real: tiene dificultades para detectar señales cuando tienen baja SNR o cuando hay mucho ruido, principalmente el causado por el viento, por lo que su rendimiento sería poco confiable en un hipotético escenario de detectar un dron lejano/silencioso al aire libre. Este problema de sensibilidad se podría resolver con micrófonos de otras características, filtros físicos para el viento, arreglos más grandes, más cantidad de arreglos y/o algoritmos más avanzados.

6.2. Posibles líneas futuras

Durante el desarrollo del trabajo surgieron distintas modificaciones y mejoras posibles; algunas se pudieron agregar mientras que otras quedaron fuera del al-

cance del proyecto. A continuación se enlistan las líneas que se podrían seguir en el futuro:

- Caracterizar de forma más exhaustiva el rendimiento del sistema: distintos ambientes, niveles de ruido y viento, las características de la propagación del sonido de las fuentes (distintos drones, detonaciones), entre otras variables.
- Desarrollar con más profundidad la localización de detonaciones de artillería. Si bien fue un objetivo considerado en este trabajo, el diseño se optimizó para drones. Un sistema pensado para este tipo de fuente debería modificar la detección, la banda de análisis, considerar otro tipo de algoritmos, el formato de los datos, tal vez elegir otros micrófonos con distinto rango dinámico.
- Implementar otros algoritmos sobre el mismo arreglo como ESPRIT, otras variantes de MUSIC y/o algoritmos de Inteligencia Artificial. Estos últimos requieren una capacidad computacional mayor pero ofrecen nuevas utilidades como la clasificación del sonido o mejor filtración de ruido, además de la localización.
- Mejorar el *frontend* de audio, la toma de datos actual limita los diseños a cuatro micrófonos y su transmisión por USB genera un cuello de botella para el *data-rate*. Se puede desarrollar o comprar arreglos que graban mayor cantidad de micrófonos de forma sincronizada, y utilizar enlaces (por cable o inalámbricos) de mayor capacidad para transmitir las señales hasta el centro de cómputo. Considerar también modificar el modelo de micrófono utilizado, INMP441 está siendo discontinuado y se puede reemplazar por modelos más modernos, con mejor rendimiento.
- Agregar una interfaz de usuario que automatice y simplifique el uso del sistema, actualmente se deben correr múltiples comandos en una terminal para ejecutar correctamente los programas.
- Pruebas con otros arreglos, de más elementos, de otras geometrías, que sean capaces de romper la ambigüedad frente-atrás, puedan soportar múltiples fuentes de forma más consistente, y que consigan mejor precisión y sensibilidad para poder implementarse en el campo.
- Implementar múltiples nodos para conseguir una ubicación completa mediante triangulación. Además de replicar el sistema se necesitaría agregar localización para cada nodo y comunicación remota con un nodo central.

- Integración con otros sistemas de detección tales como video, radar, lidar, etc.
- Medición del consumo energético. Medir el rendimiento con dos y tres fuentes simultáneas (el máximo posible teóricamente).

Apéndice A

Descomposición en autovalores

También llamada descomposición en valores propios o, en inglés, eigendecomposition o eigenvalue decomposition (EVD), es la operación clave sobre la que se basa el funcionamiento del algoritmo MUSIC.

Para una matriz cuadrada $\mathbf{A} \in \mathbb{C}^{n \times n}$, un vector $\mathbf{v} \neq 0$ es un autovector con autovalor λ si

$$\mathbf{A} \mathbf{v} = \lambda \mathbf{v} \quad (\text{A.1})$$

O sea, $\mathbf{A}$ solamente escala $\mathbf{v}$ por λ , no cambia su dirección. Son las “direcciones propias” de la transformación.

Si $\mathbf{A}$ tiene n autovectores linealmente independientes, se puede diagonalizar

$$\mathbf{A} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^{-1} \quad (\text{A.2})$$

donde las columnas de $\mathbf{V}$ son los autovectores y $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n)$.

La matriz de covarianza $\mathbf{R}_{xx}$ es hermítica (igual a su transpuesta conjugada) y semidefinida positiva (sus autovalores son iguales o mayores a cero) por definición, ya que es la esperanza estadística de una variable aleatoria por sí misma conjugada y transpuesta (ec. 2.24). Para estos casos vale el teorema espectral, una versión más fuerte de la descomposición:

$$\mathbf{R}_{xx} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^H \quad (\text{A.3})$$

donde $\mathbf{U}$ es unitaria ($\mathbf{U}^H \mathbf{U} = \mathbf{I}$), de modo que $\mathbf{U}^{-1} = \mathbf{U}^H$. Esto nos indica que los autovectores son ortonormales (ortogonales entre sí y de magnitud unitaria), forman una base ortogonal del espacio. La ortogonalidad es lo que aprovechamos al calcular el pseudoespectro.

Si se tienen M sensores e I fuentes, la matriz de dirección $\mathbf{A}$ tendrá dimensión

$(M \times I)$ y la matriz de covarianza $\mathbf{R}_{xx}$ será $(M \times M)$:

$$\mathbf{R}_{xx} = \mathbb{E}\{\mathbf{x}(t) \mathbf{x}^H(t)\} = \mathbf{A} \mathbf{R}_{ss} \mathbf{A}^H + \sigma_n^2 \mathbf{I} \quad (\text{A.4})$$

con $\mathbf{R}_{ss} = \mathbb{E}\{\mathbf{s}(t) \mathbf{s}^H(t)\}$.

Mientras las fuentes no estén correlacionadas habrá I autovalores significativamente mayores que la potencia de ruido σ_n^2 , y $M - I$ autovalores idealmente iguales a σ_n^2 . Separando los autovectores de los $M - I$ autovalores más pequeños armamos el subespacio de ruido y, sabiendo que debe ser ortogonal al espacio de la señal, podemos encontrar el vector de dirección o steering real proyectando los vectores de dirección de cada posible ángulo sobre el espacio del ruido; el que tenga una proyección menor (sea más ortogonal) será la estimación del algoritmo.

Apéndice B

Dispositivos y herramientas utilizadas

B.1. Dispositivos utilizados

B.1.1. Micrófonos INMP441

Micrófonos omnidireccionales de salida digital y baja potencia. El módulo “breakout” (el chip montado en una placa con una resistencia para filtrar el ruido de alimentación, un capacitor de desacople y conectores para sus señales) cuenta con un sensor MEMS (micro-electro-mecánico), acondicionamiento de la señal, un conversor analógico-digital, filtros anti aliasing y una interfaz I2S de 24 bits. Este tipo de micrófonos son cada vez más comunes en los arreglos acústicos principalmente por su buen y consistente desempeño, su bajo costo y tamaño reducido, además de simplificar el diseño de la cadena de procesamiento al digitalizar y filtrar la señal (Ramamonjy et al., 2018; Riabko et al., 2024; Zhang et al., 2014). La mayoría de sistemas previos estudiados, algunos desarrollados en la Sección 2.5, utilizan este tipo de micrófono.

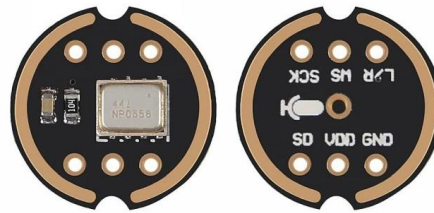


Figura B.1: Módulo breakout del INMP441.

El modelo INMP441 cuenta con una Relación Señal-Ruido (SNR) de 61 dBA (decibeles ponderados “A”, considera la percepción del oído humano a distintas frecuencias), una sensibilidad de -26 dBFS (decibeles Full Scale) y una respuesta plana en frecuencia desde 60 Hz hasta 15 kHz, por lo que nuestra banda de trabajo (200–2400 Hz) entra cómodamente.

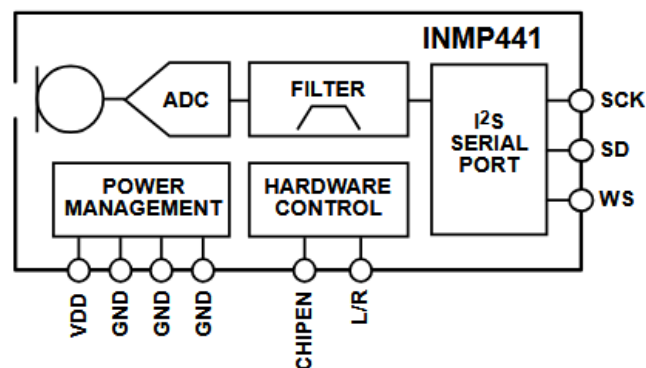


Figura B.2: Diagrama en bloques del INMP441.

La señal digital de salida del micrófono está dada por una interfaz I2S (Inter-IC Sound, sonido inter circuito integrado), un protocolo de bus serie diseñado para la transmisión de audio digital entre circuitos. El bus consiste de al menos tres líneas:

1. Línea del reloj serial de bit (SCK).
2. Línea de selección de palabra (WS).
3. Línea de datos de salida (SD).

El formato de los datos de salida de I2S es PCM (Pulse Code Modulation), codificación binaria de complemento a dos de 24 bits. El protocolo permite utilizar dos canales en la línea de datos (audio estéreo) mediante el estado de WS y la configuración del pin L/R de cada micrófono de la siguiente manera:

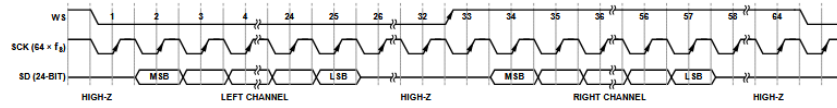


Figure 8. Stereo-Output I2S Format

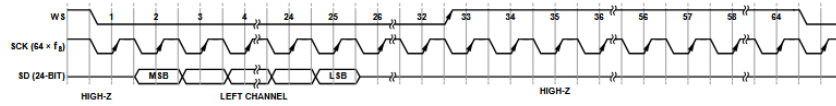


Figure 9. Mono-Output I2S Format Left Channel (L/R = 0)

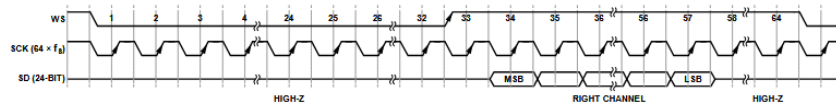


Figure 10. Mono-Output I2S Format Right Channel (L/R = 1)

Figura B.3: Output I2S estéreo (arriba) y mono (abajo).

Se menciona que utilizar una configuración mono no aumenta la frecuencia de muestreo del micrófono: aunque los datos salgan en serie de forma alternada están sincronizados por la misma señal WS, podemos entonces considerar que las muestras son simultáneas, propiedad imprescindible para realizar los algoritmos de DOA. Para enviar los datos a la unidad de procesamiento se aprovechó la configuración estéreo para transmitir la señal de los micrófonos de a pares, con la siguiente configuración:

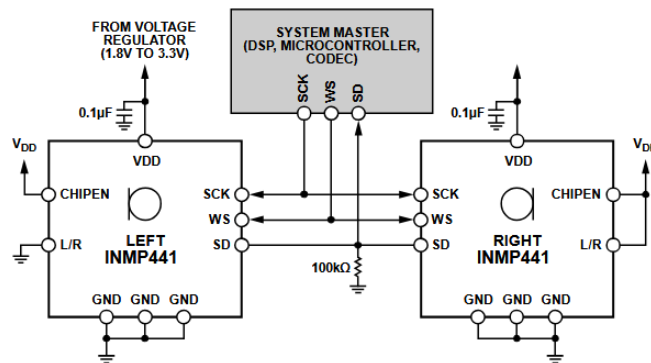


Figura B.4: Configuración estéreo de dos micrófonos INMP441.

La fabricación del modelo específico INMP441 fue descontinuada durante el desarrollo de este trabajo, pero es fácilmente reemplazable por otro modelo que utilice el protocolo I2S, teniendo que modificar tal vez la placa PCB, el cableado, y/o algunos valores de la configuración.

B.1.2. ESP32

El frontend para la adquisición de datos es un ESP32, un microcontrolador de bajo costo que cuenta con procesador de doble núcleo, múltiples periféricos, conectividad inalámbrica y serial USB. La característica que determina el uso del ESP32 para este proyecto es que cuenta con dos buses I2S independientes, lo que permite capturar los cuatro micrófonos del arreglo de forma sincronizada, y un puerto USB para transmitir los cuatro canales hasta la Raspberry Pi.

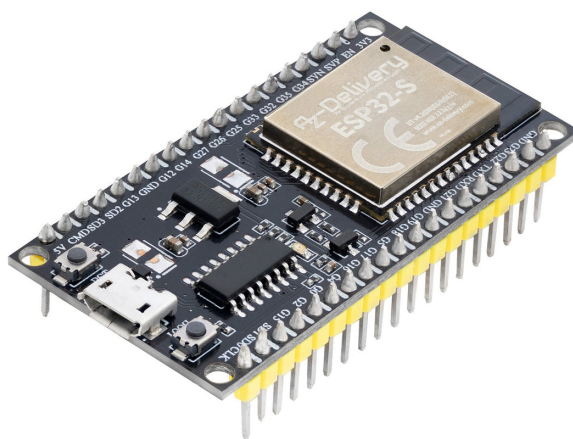


Figura B.5: Placa ESP32.

B.1.3. Raspberry Pi 3A+

La unidad de procesamiento principal del sistema, la Raspberry Pi 3A+, es una computadora de placa única con un procesador quad-core 1,4 GHz de 64 bits, 512 MB de memoria SDRAM, conectividad inalámbrica (WiFi y Bluetooth/BLE), puerto USB y 40 pines GPIO de entrada/salida. Cuenta con un sistema operativo completo, en este caso Raspberry Pi OS, un port de Debian Trixie (Linux). El proyecto es compatible con otros modelos. La placa ejecuta todo el procesamiento de las señales, la escritura del registro y el control de los servomotores con scripts de Python, utilizando NumPy para los cálculos más intensivos.



Figura B.6: Placa Raspberry Pi 3A+.

B.1.4. Dron DJI Mini 4 Pro

Utilizado para probar el sistema, este dron cámara está diseñado para ser recreativo, portable y ligero; pesa menos de 249 gramos y cuenta con múltiples sistemas de asistencia para el vuelo. Pero lo más relevante para este proyecto es su perfil de ruido: es uno de los drones más silenciosos del mercado comercial. Es pequeño, liviano y sus hélices están diseñadas para reducir el ruido; a una altura de 1,5 metros el fabricante indica un ruido máximo, en condiciones específicas, de 81 dBA, pero otras mediciones indican que puede estar en los 67 dBA, niveles similares a los de una conversación (Noise Levels And Stealth Features Of DJI Mini 4 Pro, s/f). Los niveles de ruido se reducen rápidamente al aumentar la distancia, siendo prácticamente imperceptible alrededor de los 50 metros, especialmente si hay mucho viento.

Estas características dificultan la evaluación del sistema al mismo tiempo que evidencian las limitaciones intrínsecas del método de detección. A medida que el avance en la fabricación de drones continúe, serán necesarios sistemas más sensibles y complejos para poder detectarlos.



Figura B.7: Dron DJI Mini 4 Pro.

B.2. Herramientas utilizadas

B.2.1. Matlab

Un entorno de desarrollo integrado (IDE) y computación numérica que cuenta con un lenguaje de programación propio y herramientas especiales pensadas para manipular y graficar matrices, variables, funciones, implementar algoritmos y otras prestaciones. Fue el software utilizado para las simulaciones (Capítulo 3) y para graficar algunas de las figuras presentadas en el trabajo. Se pueden ampliar las capacidades del programa base mediante lo que se conoce como toolboxes; para este trabajo se utilizó el Phased Array System Toolbox o “arreglos de fase”, que incluye herramientas para el diseño y la simulación de arreglos de antenas o sensores y algoritmos de estimación de DOA.

B.2.2. ArduinoIDE

Software de código abierto utilizado para programar, compilar y subir código a la placa ESP32. Originalmente diseñado para trabajar con placas Arduino, se puede usar para numerosos microcontroladores compatibles. Dentro del IDE se programan “sketches”, programas en C++ que se compilan y suben a las placas mediante USB. Cuenta con herramientas para facilitar el flujo de trabajo como el Serial Plotter, que permite visualizar datos de la placa en tiempo real, o el administrador de librerías.

B.2.3. Visual Studio Code (VS Code)

Editor de código sobre el que se programaron los scripts principales del proyecto. Se aprovechó la opción de conectarse mediante SSH a la Raspberry Pi para editar directamente sobre sus archivos en lugar de editarlos localmente y transferirlos.

B.2.4. EasyEDA

Herramienta de software de Automatización de Diseño Electrónico (EDA) basada en la nube, permite diseñar de forma simple esquemas y placas de circuito impreso (PCB) directamente desde el navegador web; además permite exportar archivos Gerber, el estándar para fabricar las PCB. Se utilizó para diseñar las placas de los dos sistemas implementados (Apéndice C).

B.2.5. Fresadora CNC

Para fabricar las plaquetas diseñadas, finalmente, se utilizó la Fresadora CNC (Control Numérico Computarizado) del laboratorio de prototipado electrónico de la Universidad Nacional de Río Negro. Esta máquina graba las pistas y perfora las vías en una placa de cobre a partir del archivo Gerber generado en EasyEDA, luego de lo cual solamente resta soldar los componentes para completar la implementación.

Apéndice C

Esquemáticos y PCBs

C.1. Esquemáticos

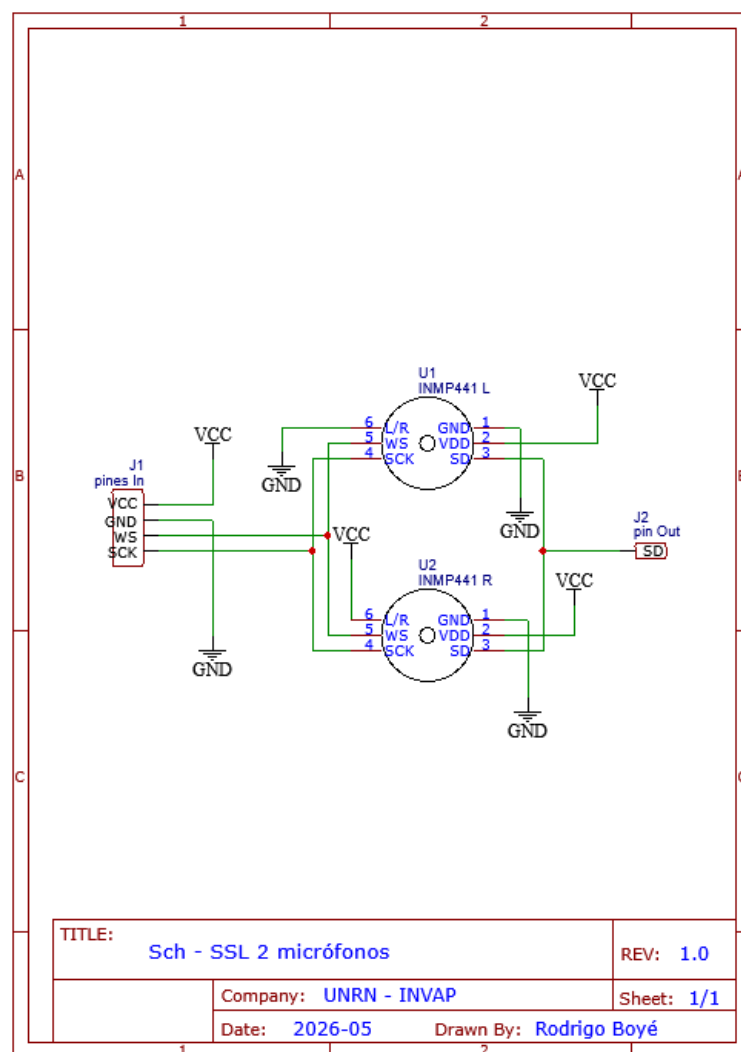


Figura C.1: Plano esquemático del sistema de dos micrófonos.

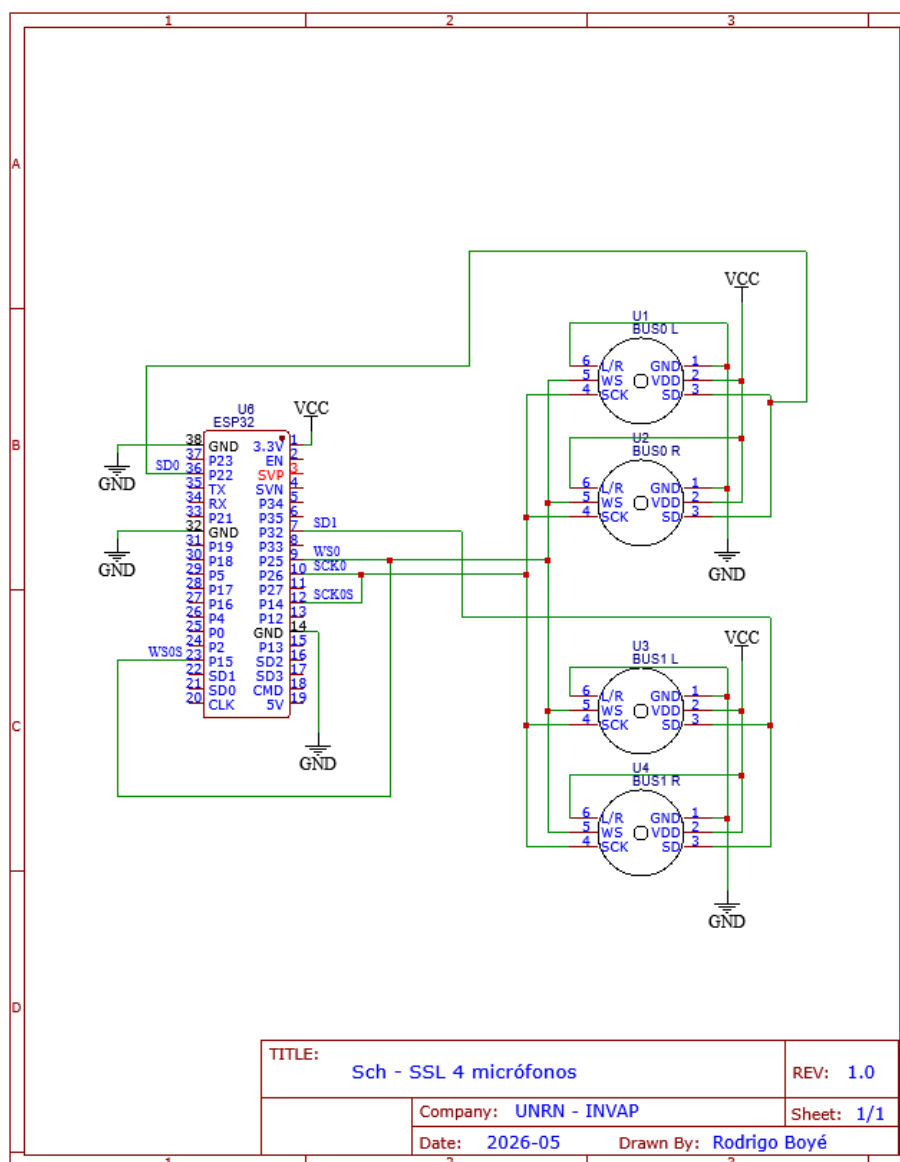
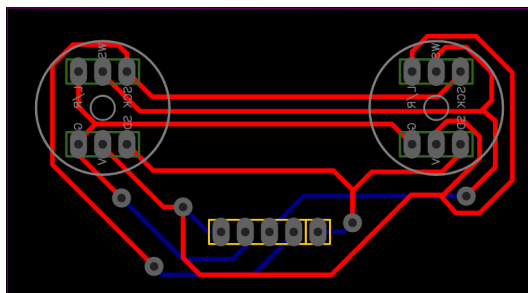
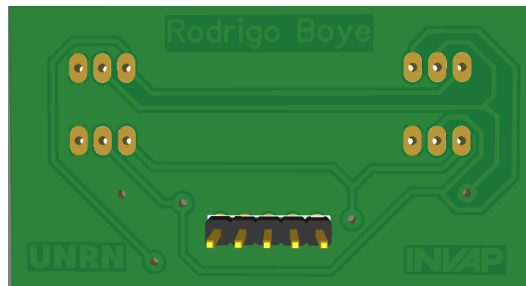


Figura C.2: Plano esquemático del sistema de cuatro micrófonos.

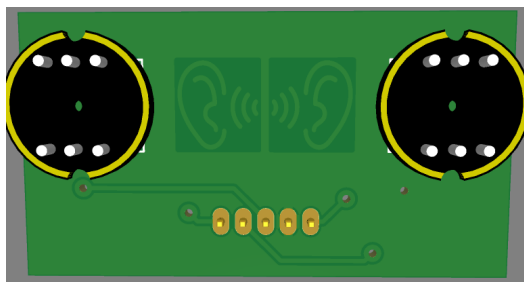
C.2. PCBs



(a) Plano

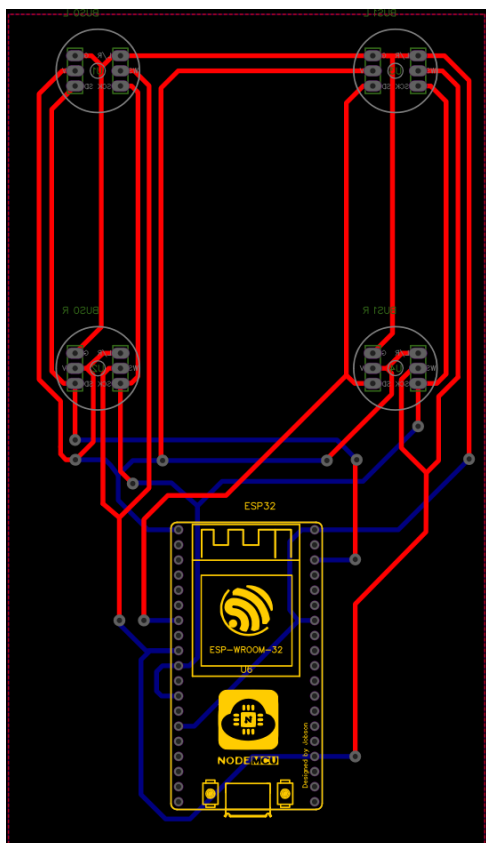


(b) Render trasero

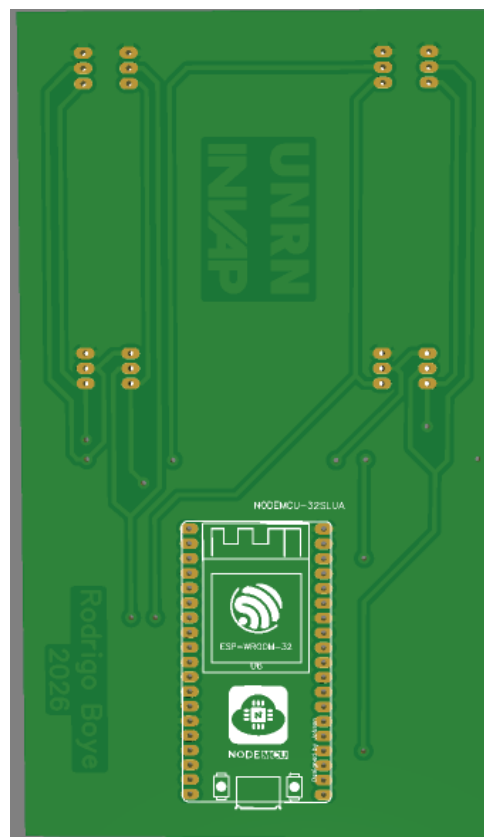


(c) Render frontal

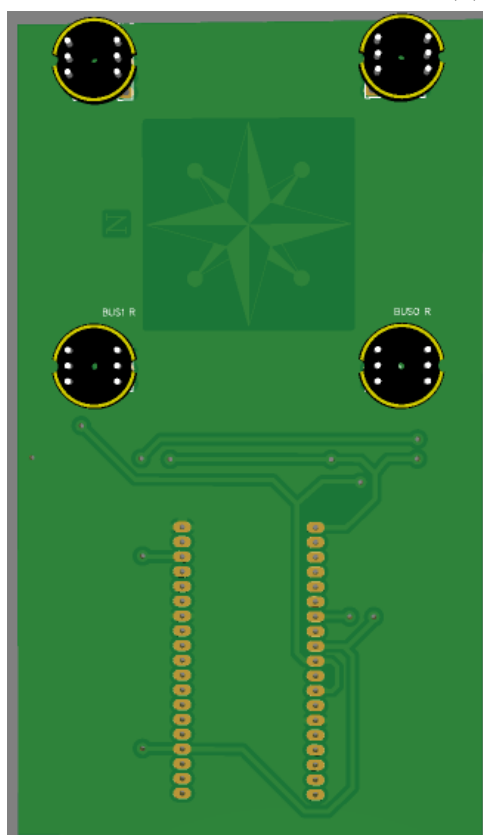
Figura C.3: PCB del sistema de dos micrófonos.



(a) Plano



(b) Render trasero



(c) Render frontal

Figura C.4: PCB del sistema de cuatro micrófonos.

Apéndice D

Programas utilizados en el diseño final

Este apéndice describe, mediante pseudocódigo, la lógica de cada uno de los programas que componen el sistema de cuatro micrófonos. El objetivo no es reproducir el código fuente línea por línea, sino documentar de forma clara qué hace cada módulo, por qué, y cómo se relaciona con los demás. El código completo, junto con los archivos de los planos, algunas de las referencias y videos de los sistemas en funcionamiento se pueden encontrar en el repositorio del proyecto (<https://github.com/rodriboye/PFI-localizacion-acustica>). El pseudocódigo usa una notación genérica (asignación con $\leftarrow$, comentarios con $\#$) e independiente del lenguaje.

El sistema se reparte entre dos dispositivos: el ESP32, que actúa como frontend de adquisición y captura los cuatro canales de audio, y la Raspberry Pi, que ejecuta el resto de la cadena (recepción, detección, estimación de dirección de arribo, registro y control de actuadores). La arquitectura de módulos es la siguiente:



El flujo de datos es unidireccional: cada bloque de audio atraviesa la cadena `audio_input` $\rightarrow$ `detector` $\rightarrow$ `spectral_gate` $\rightarrow$ `doa_engine`, y solo se ejecutan las etapas costosas (estimación de DOA) si las etapas de decisión previas lo autorizan. Esta arquitectura modular permite probar cada parte por separado y desactivar etapas individualmente para depuración.

D.1. ESP32_audio_frontend

Función. Programa embebido que corre en el ESP32. Su única responsabilidad es capturar de forma **sincronizada** los cuatro micrófonos y transmitirlos hacia la Raspberry Pi por el puerto serie USB. Para garantizar que las cuatro muestras correspondan al mismo instante (condición imprescindible para estimar la DOA) se configuran los dos buses I2S del ESP32 en modo **maestro/esclavo con reloj compartido**: el bus 0 genera las señales de reloj (CLK) y selección de palabra (WS), y el bus 1 se limita a seguirlas. Así se evita cualquier deriva entre dos relojes independientes.

Los datos se envían agrupados en **paquetes (“hops”)** delimitados por bytes de sincronismo y numerados con un contador, de modo que el receptor pueda realinear la trama y detectar paquetes perdidos.

```
# --- Configuración inicial (se ejecuta una vez) ---
procedimiento setup():
    inicializar_bus_I2S(bus0, modo = MAESTRO, fs = FS, bits = 16) # genera CLK y WS
    inicializar_bus_I2S(bus1, modo = ESCLAVO, reloj = bus0)      # sigue a bus0
    # bus0 -> micrófonos {0,1} * bus1 -> micrófonos {2,3}
    inicializar_USB_serial(baudrate = BAUDRATE)                 # p. ej. 921600
    contador <- 0

# --- Bucle principal (se ejecuta indefinidamente) ---
procedimiento loop():
    mientras verdadero:
        # 1. Leer HOP_SIZE muestras de cada bus, en simultáneo
        crudo0 <- leer_I2S(bus0, HOP_SIZE) # muestras entrelazadas mic0/mic1
        crudo1 <- leer_I2S(bus1, HOP_SIZE) # muestras entrelazadas mic2/mic3

        # 2. Desentrelazar y reordenar a 4 canales alineados por muestra
        bloque <- intercalar_canales(crudo0, crudo1) # forma: [HOP_SIZE][4] int16

        # 3. Empaquetar el hop con delimitadores y número de secuencia
        paquete <- []
        paquete.append(0xAA) # byte de sincronismo de inicio
        paquete.append(byte_alto(contador), byte_bajo(contador)) # contador de 16 bits
        paquete.append(serializar(bloque)) # HOP_SIZE x 4 x 2 bytes
        paquete.append(0x55) # byte de sincronismo de fin

        # 4. Enviar y avanzar el contador (con envoltorio a 16 bits)
        escribir_USB_serial(paquete)
        contador <- (contador + 1) mod 65536
```

Notas de diseño. La frecuencia de muestreo y la resolución en bits se eligen para que el caudal de datos no supere el límite del enlace USB ($4 \text{ canales} \times 2 \text{ bytes} \times \text{FS}$). El contador de dos bytes es lo que permite al receptor cuantificar la pérdida de paquetes sin necesidad de un canal de retorno.

D.2. config

Función. Módulo (sin lógica ejecutable) que centraliza **todos los parámetros** del sistema en un único lugar. Separar la configuración del código permite ajustar el comportamiento (geometría, banda de trabajo, umbrales, algoritmo, actuadores) sin tocar la implementación, y deja documentado en un solo archivo el estado con el que se corrió cada prueba.

```
# --- Adquisición y enlace ---
FS          <- frecuencia de muestreo [Hz]
BITS        <- resolución por muestra [bits]
HOP_SIZE    <- muestras por paquete/bloque de procesamiento
N_CANALES   <- 4
BAUDRATE    <- velocidad del puerto serie USB
PUERTO_SERIE <- identificador del puerto (p. ej. /dev/ttyUSB0)

# --- Geometría del arreglo ---
POSICIONES_MIC <- lista de coordenadas (x, y, z) de cada micrófono [m]
LADO_ARREGLO   <- separación entre micrófonos [m]
ANGULO_INCLIN  <- inclinación física del arreglo [grados] # se suma a la elevación
VEL_SONIDO     <- 331 + 0.6 x TEMPERATURA [m/s]           # recalculable por clima

# --- Banda de trabajo y grilla de escaneo ---
FREC_MIN, FREC_MAX <- límites de la banda útil [Hz]      # p. ej. 200 y 2400
GRILLA_ACIMUT     <- puntos de acimut en espacio-u
GRILLA_ELEVACION  <- puntos de elevación en espacio-u

# --- Detección por energía (detector) ---
PISO_RUIDO_FIJO   <- valor de piso de ruido (si no se mide en vivo)
MEDIR_PISO_INICIO <- booleano: medir el piso al arrancar
UMBRAL_EVENTO     <- múltiplo del piso para iniciar un evento (onset)
UMBRAL_SALIDA     <- múltiplo del piso para cerrar el evento (< onset, histéresis)
FRAMES_CONFIRMAR  <- frames requeridos para confirmar (ONSET -> ACTIVE)
FRAMES_COOLDOWN   <- frames de espera tras un evento

# --- Detección espectral (spectral_gate) ---
BPF_MIN, BPF_MAX  <- rango esperado de la frecuencia de paso de pala [Hz]
N_ARMONICOS_MIN   <- armónicos con SNR suficiente para confirmar dron
SNR_MIN_ARMONICO  <- umbral de SNR por armónico [dB]
FRACC_ENERGIA_MIN <- fracción mínima de energía en el peine armónico
USAR_SPECTRAL_GATE <- booleano: activar/desactivar el filtro de dron

# --- Estimación de DOA (doa_engine) ---
ALGORITMO         <- MUSIC | SRP-PHAT
ALPHA_PROMEDIO    <- factor de promediado temporal exponencial de la covarianza
N_FUENTES         <- número de fuentes asumido (típicamente 1)

# --- Actuadores y registro ---
USAR_SERVO        <- booleano
DEADZONE          <- mínimo cambio angular para mover el servo [grados]
N_PROMEDIO_SERVO  <- estimaciones promediadas antes de mover el servo
ARCHIVO_LOG       <- ruta del archivo .csv de eventos
MODO_DEFECTO     <- seguimiento | evento
```

D.3. audio_input

Función. Primer módulo de la cadena en la Raspberry Pi. Lee el flujo de bytes que llega por USB, **realinea la trama** buscando los bytes de sincronismo, valida cada paquete con el contador de secuencia (para detectar pérdidas), y **desempaqueta** las muestras a una matriz de cuatro canales lista para procesar. Aísla al resto del sistema de los detalles del enlace serie: los módulos siguientes reciben bloques limpios sin saber cómo llegaron.

```
procedimiento leer_bloque():
    # 1. Buscar el byte de sincronismo de inicio (resincronización robusta)
    repetir:
        b <- leer_byte(PUERTO_SERIE)
    hasta que b == 0xAA

    # 2. Leer el número de secuencia
    contador <- leer_uint16(PUERTO_SERIE)

    # 3. Leer el bloque de muestras (HOP_SIZE x 4 canales x 2 bytes)
    crudo <- leer_bytes(PUERTO_SERIE, HOP_SIZE x 4 x 2)

    # 4. Verificar el byte de sincronismo de fin
    fin <- leer_byte(PUERTO_SERIE)
    si fin != 0x55:
        # Trama corrupta: descartar y volver a sincronizar en la próxima llamada
        registrar_paquete_corrupto()
        retornar NULO

    # 5. Control de continuidad: detectar paquetes perdidos
    si contador != (contador_previo + 1) mod 65536:
        registrar_paquetes_perdidos(contador - contador_previo)
    contador_previo <- contador

    # 6. Convertir bytes a enteros con signo y separar en canales
    bloque <- desempaquetar_int16(crudo)          # forma [HOP_SIZE][4]
    señales <- separar_por_canal(bloque)         # 4 vectores de HOP_SIZE muestras

    retornar señales
```

Notas de diseño. La resincronización por búsqueda del byte 0xAA hace que el sistema se recupere solo después de cualquier corrupción o arranque a mitad de trama, sin quedar desalineado permanentemente. La verificación del contador convierte una eventual pérdida de datos en un dato observable (útil para diagnosticar cuellos de botella del enlace) en lugar de un error silencioso.

D.4. detector

Función. Detector por **energía**. Decide si en el bloque actual hay una señal que valga la pena procesar o si es solo silencio/ruido de fondo. Su propósito

es evitar el gasto de cómputo de correr la estimación de DOA sobre ruido. Se implementa como una **máquina de cuatro estados** con **histéresis** (el umbral para abrir un evento es mayor que el umbral para cerrarlo), lo que evita que el sistema entre y salga del estado de detección ante fluctuaciones del nivel alrededor del umbral.

Estados: IDLE (silencio) → ONSET (posible evento, en confirmación) → ACTIVE (evento confirmado) → COOLDOWN (espera tras el cierre).

```
# --- Inicialización ---
procedimiento init_detector():
  si MEDIR_PISO_INICIO:
    piso_ruido <- medir_energia_promedio(varios_bloques_de_silencio)
  sino:
    piso_ruido <- PISO_RUIDO_FIJO
  estado <- IDLE
  contador_frames <- 0
  limpiar(matriz_covarianza)      # se irá llenando durante ONSET

# --- Procesamiento de un bloque ---
procedimiento procesar(señales):
  energia <- energia_media(señales)          # p. ej. media de la potencia
  umbral_on <- UMBRAL_EVENTO x piso_ruido
  umbral_off <- UMBRAL_SALIDA x piso_ruido    # umbral_off < umbral_on

  segun estado:

    caso IDLE:
      si energia > umbral_on:
        estado <- ONSET
        contador_frames <- 0
        retornar {evento: NO}

    caso ONSET:
      # Confirmar que la energía se sostiene durante varios frames.
      # Durante la confirmación se acumula ya la matriz de covarianza,
      # así el doa_engine arranca con datos apenas se confirme el evento.
      acumular_covarianza(señales, matriz_covarianza)
      si energia > umbral_on:
        contador_frames <- contador_frames + 1
        si contador_frames >= FRAMES_CONFIRMAR:
          estado <- ACTIVE
          retornar {evento: SI, nuevo: SI}
      sino:
        estado <- IDLE          # falsa alarma, se descarta
        retornar {evento: NO}

    caso ACTIVE:
      si energia > umbral_off:
        retornar {evento: SI, nuevo: NO}    # el evento continúa
      sino:
        estado <- COOLDOWN
        contador_frames <- 0
        retornar {evento: NO, cerrar: SI}    # se cierra el evento

    caso COOLDOWN:
      # Espera para no volver a disparar con la cola del mismo evento.
```

```

contador_frames <- contador_frames + 1
si contador_frames >= FRAMES_COOLDOWN:
    estado <- IDLE
retornar {evento: NO}

```

Notas de diseño. La histéresis y el estado COOLDOWN son los que dan robustez: sin ellos, un sonido cuyo nivel oscila cerca del umbral generaría una ráfaga de eventos falsos de apertura y cierre. Acumular la covarianza ya en ONSET reduce la latencia efectiva del `doa_engine`.

D.5. spectral_gate

Función. Segunda etapa de detección, **específica para drones**. El detector por energía se dispara con cualquier sonido fuerte; este módulo lo filtra dejando pasar solo señales con la **firma acústica** característica de un rotor: un tono a la frecuencia de paso de pala (BPF) más una serie de armónicos (un “peine” en el espectro). Para estimar la BPF usa el **Espectro de Productos Armónicos (HPS)**, que refuerza las frecuencias que tienen armónicos y atenúa el ruido de banda ancha. Se puede desactivar por configuración para probar el sistema con otros sonidos.

```

procedimiento es_dron(señales):
    si no USAR_SPECTRAL_GATE:
        retornar VERDADERO          # etapa desactivada: deja pasar todo

    # 1. Espectro de magnitud (se usa un canal o la suma de los cuatro)
    espectro <- |FFT(mezclar_canales(señales))|

    # 2. Estimar la BPF con el Espectro de Productos Armónicos (HPS)
    #   HPS(f) = producto de la magnitud en f y sus múltiplos enteros.
    #   Una frecuencia con armónicos verdaderos se refuerza; el ruido no.
    para cada f en [BPF_MIN .. BPF_MAX]:
        HPS[f] <- espectro[f] x espectro[2f] x espectro[3f] x ...   # hasta N armónicos
    BPF <- argmax(HPS)

    # 3. Medir el SNR de cada armónico candidato (múltiplos de la BPF)
    #   El nivel de referencia de ruido es el mayor entre la mediana local
    #   del espectro y el piso global, para no sobreestimar el SNR.
    armónicos_validos <- 0
    hay_armónico_en_banda <- FALSO
    energía_peine <- 0
    para k en 1, 2, 3, ...:
        fk <- k x BPF
        ruido_ref <- max(mediana_local(espectro, fk), piso_global)
        snr_k <- 10*log10( espectro[fk]^2 / ruido_ref^2 )
        si snr_k >= SNR_MIN_ARMONICO:
            armónicos_validos <- armónicos_validos + 1
            energía_peine <- energía_peine + espectro[fk]^2
        si FREC_MIN <= fk <= FREC_MAX:
            hay_armónico_en_banda <- VERDADERO

```

```

# 4. Criterios de confirmación (deben cumplirse todos)
fraccion <- energia_peine / energia_total(espectro)
confirmado <- (armonicos_validos >= N_ARMONICOS_MIN)
               Y (hay_armonico_en_banda)                # al menos uno útil para MUSIC
               Y (fraccion >= FRACC_ENERGIA_MIN)

retornar confirmado

```

Notas de diseño. Exigir simultáneamente varios armónicos, que al menos uno caiga dentro de la banda de trabajo del estimador y que el peine concentre una fracción mínima de la energía total evita falsos positivos con tonos aislados o ruido de banda ancha. Este módulo también habilita una posible optimización del `doa_engine`: correr la estimación solo en los bins donde hay componentes armónicos, mejorando el SNR y bajando el costo.

D.6. `doa_engine`

Función. Núcleo del sistema: estima la **dirección de arribo** (acimut y elevación) del bloque de audio. Implementa dos algoritmos intercambiables: **MUSIC** de banda ancha incoherente (motor principal, alta resolución para fuentes sostenidas y armónicas como un dron) y **SRP-PHAT** (alternativa más robusta para fuentes impulsivas o entornos reverberantes). La salida es el par de ángulos estimados y una **medida de confianza**.

Los vectores de dirección (steering) para cada punto de la grilla de escaneo y cada banda de frecuencia se **precalculan una sola vez** al iniciar y se guardan en una tabla, de modo que durante la operación en tiempo real solo se consultan.

```

# --- Precálculo (una vez, al iniciar) ---
procedimiento init_doa():
  para cada banda f en [FREC_MIN .. FREC_MAX]:
    para cada (u_az, u_el) en la grilla en espacio-u:      # u = sin(ángulo)
      tabla_steering[f][u_az][u_el] <- vector_steering(POSICIONES_MIC, f, u_az, u_el)
  limpiar(covarianza_por_banda)      # una matriz por bin de frecuencia

```

MUSIC (banda ancha incoherente)

```

procedimiento estimar_MUSIC(señales):
  # 1. Pasar los cuatro canales a frecuencia
  X <- FFT_por_canal(señales)      # X[canal][bin]

  pseudoespectro_acumulado <- ceros(grilla)

  # 2. Recorrer los bins dentro de la banda útil
  para cada bin f en [FREC_MIN .. FREC_MAX]:
    x_f <- vector_de_canales(X, f)      # 4 valores complejos

```

```
# 2.a Actualizar la covarianza espacial de esta banda con
#   promediado temporal exponencial (memoria decreciente).
R[f] <- ALPHA_PROMEDIO x R[f] + (1 - ALPHA_PROMEDIO) x (x_f * x_f^H)

# 2.b Descomponer en autovalores (ver Apéndice A) y separar subespacios
(autovalores, autovectores) <- eigendecomposicion(R[f])
Vn <- autovectores asociados a los (M - N_FUENTES) autovalores menores # ruido

# 2.c Proyectar cada vector de dirección sobre el subespacio de ruido.
#   La verdadera dirección es ortogonal al ruido -> proyección ~ 0 -> pico.
para cada (u_az, u_el) en la grilla:
    a <- tabla_steering[f][u_az][u_el]
    P[u_az][u_el] <- 1 / || Vn^H * a ||^2
pseudoespectro_acumulado <- pseudoespectro_acumulado + P

# 3. Buscar el pico 2D y refinarlo por interpolación parabólica
(u_az*, u_el*) <- argmax_2D(pseudoespectro_acumulado)
(u_az*, u_el*) <- interpolacion_parabolica(pseudoespectro_acumulado, u_az*, u_el*)

# 4. Convertir de espacio-u a grados y aplicar la inclinación física
acimut <- arcsin(u_az*)
elevacion <- arcsin(u_el*) + ANGULO_INCLIN

# 5. Confianza: relación en dB entre el pico y la mediana del pseudoespectro
confianza <- 10*log10( pico / mediana(pseudoespectro_acumulado) )

retornar {acimut, elevacion, confianza}
```

SRP-PHAT (alternativa)

```
procedimiento estimar_SRP_PHAT(señales):
    X <- FFT_por_canal(señales)

    # Precalcular GCC-PHAT de cada par de micrófonos (i, j)
    para cada par (i, j):
        Gij <- X[i] x conjugado(X[j])
        Gij <- Gij / |Gij| # ponderación PHAT: solo la fase
        gcc[i][j] <- IFFT(Gij) # correlación cruzada generalizada

    # Escanear la grilla: para cada dirección, sumar la GCC de cada par
    # evaluada en el retardo teórico que implicaría esa dirección.
    para cada (u_az, u_el) en la grilla:
        suma <- 0
        para cada par (i, j):
            tau <- retardo_teorico(POSICIONES_MIC[i], POSICIONES_MIC[j], u_az, u_el)
            suma <- suma + gcc[i][j][tau]
        SRP[u_az][u_el] <- suma

    (u_az*, u_el*) <- argmax_2D(SRP)
    acimut <- arcsin(u_az*)
    elevacion <- arcsin(u_el*) + ANGULO_INCLIN
    confianza <- 10*log10( pico(SRP) / mediana(SRP) )
    retornar {acimut, elevacion, confianza}
```

Notas de diseño. El promediado temporal exponencial de la covarianza es el que permite que MUSIC afine su estimación sobre fuentes continuas (más memo-

ria = más estable; menos memoria = reacciona más rápido). La grilla en espacio-u concentra los puntos de escaneo donde el arreglo tiene mayor sensibilidad (cerca del broadside) y no desperdicia resolución en los bordes. La interpolación parabólica entrega una resolución mayor que el paso de la grilla sin aumentar el número de puntos escaneados.

D.7. main

Función. Programa orquestador que corre en la Raspberry Pi. Inicializa todos los módulos, elige el **modo de operación** (“seguimiento” o “evento”), y ejecuta el bucle principal que encadena la cadena de procesamiento y dispara las salidas del sistema: **visualización** en la terminal, **registro** de eventos en un archivo .csv y **control de los servomotores** que apuntan a la dirección estimada.

Los dos modos se diferencian en el comportamiento frente a la señal: el modo **seguimiento** usa MUSIC y acumula datos para afinar la estimación sobre fuentes sostenidas; el modo **evento** usa SRP-PHAT por defecto y responde de forma inmediata, sin esperar la confirmación del detector.

```
procedimiento main():
    # --- Inicialización ---
    cargar(config)
    modo <- argumento_de_ejecucion o MODO_DEFECTO      # seguimiento | evento
    abrir_puerto_serie(PUERTO_SERIE, BAUDRATE)
    init_detector()
    init_doa()
    log <- abrir_csv(ARCHIVO_LOG, columnas =
        [t_inicio, t_fin, acimut, elevacion, confianza, energia])
    si USAR_SERVO: init_servos()
    evento_abierto <- FALSO
    buffer_servo <- []

    # --- Bucle principal ---
    mientras verdadero:
        # 1. Adquisición
        señales <- audio_input.leer_bloque()
        si señales == NULO: continuar      # paquete corrupto: saltar

        # 2. Detección por energía
        d <- detector.procesar(señales)
        si no d.evento:
            si evento_abierto: cerrar_evento(log)      # se acaba de cerrar el evento
            evento_abierto <- FALSO
            display_silencio()
            continuar

        # 3. Detección espectral (filtro de dron)
        si no spectral_gate.es_dron(señales):
            continuar      # sonido fuerte pero no es un dron
```

```

# 4. Estimación de DOA (algoritmo según modo)
si modo == seguimiento:
    r <- doa_engine.estimated_MUSIC(señales)
sino: # modo evento
    r <- doa_engine.estimated_SRP_PHAT(señales)

# 5. Salidas
# 5.a Visualización en tiempo real (barra ASCII en la terminal)
mostrar_barra(r.acimut, r.elevacion, r.confianza)

# 5.b Registro: acumular mientras el evento esté activo
si no evento_abierto:
    iniciar_evento(log, t_actual)
    evento_abierto <- VERDADERO
acumular_en_evento(r.acimut, r.elevacion, r.confianza, energia_media(señales))

# 5.c Servomotores con suavizado
si USAR_SERVO:
    buffer_servo.append(r.acimut, r.elevacion)
    si tamaño(buffer_servo) >= N_PROMEDIO_SERVO:
        objetivo <- promedio(buffer_servo) # filtra el jitter
        si distancia angular(objetivo, posicion_actual) > DEADZONE:
            mover_servos(objetivo) # zona muerta
            esperar_llegada()
            detach_PWM() # evita jitter por SW
        buffer_servo <- []

```

Notas de diseño. El orden de la cadena (energía primero, firma espectral después y recién entonces la estimación de DOA) hace que los cálculos caros solo se ejecuten cuando hay razones para hacerlo. El suavizado del servo combina tres mecanismos complementarios: promediado (filtra el ruido de estimación), zona muerta (evita movimientos irrelevantes) y desconexión del PWM al llegar a la posición (elimina el temblor causado por la imprecisión del ancho de pulso generado por software, ya que el motor mantiene la posición por fricción interna).

Capturas de pantalla. La salida del programa se muestra en la terminal, se incluye un ejemplo de una estimación en tiempo real:

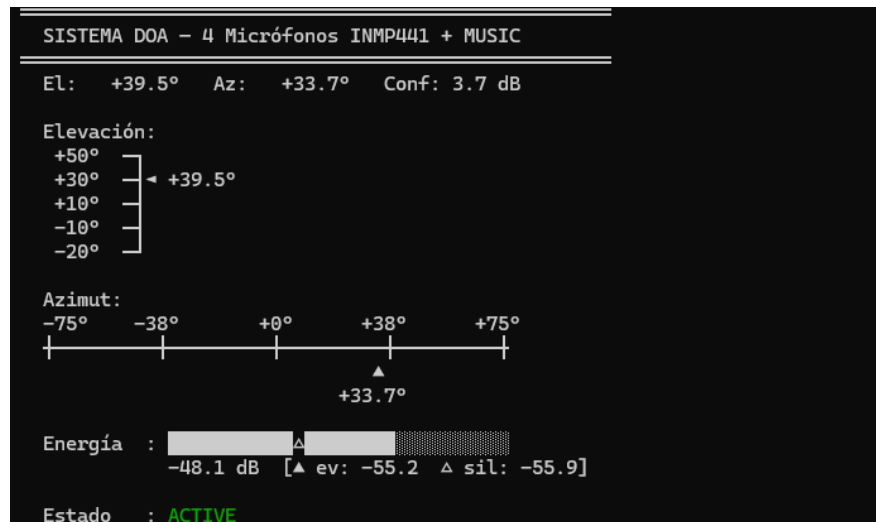


Figura D.1: Captura de pantalla de la interfaz del sistema por terminal

Bibliografia

- Belloch, J. A., Badia, J. M., Igual, F. D., & Cobos, M. (2019). Practical Considerations for Acoustic Source Localization in the IoT Era: Platforms, Energy Efficiency, and Performance. *IEEE Internet of Things Journal*, 6(3), 5068–5079. doi: 10.1109/JIOT.2019.2895742
- Brandstein, M., & Ward, D. (Eds.). (2001). *Microphone Arrays: Signal Processing Techniques and Applications*. Springer Berlin Heidelberg. <https://doi.org/10.1007/978-3-662-04619-7>
- Case, E. E., Zelnio, A. M., & Rigling, B. D. (2008). Low-Cost Acoustic Array for Small UAV Detection and Tracking. *2008 IEEE National Aerospace and Electronics Conference*, 110–113. doi: 10.1109/NAECON.2008.4806528
- Chen, Z. D., Gokeda, G., & Yu, Y. (2010). *Introduction to direction-of-arrival estimation*. Artech House.
- Cobos, M., Antonacci, F., Alexandridis, A., Mouchtaris, A., & Lee, B. (2017). A Survey of Sound Source Localization Methods in Wireless Acoustic Sensor Networks. *Wireless Communications and Mobile Computing*, 2017, 1–24. doi: 10.1155/2017/3956282
- Fabregat, G., Belloch, J. A., Badia, J. M., & Cobos, M. (2020). Design and Implementation of Acoustic Source Localization on a Low-Cost IoT Edge Platform. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 67(12), 3547–3551. doi: 10.1109/TCSII.2020.2986296
- Friedlander, B. (2009). *Classical and modern direction-of-arrival estimation*. Academic.
- Jekaterýńczuk, G., & Piotrowski, Z. (2023). A Survey of Sound Source Localization and Detection Methods and Their Applications. *Sensors*, 24(1), 68. doi: 10.3390/s24010068
- Knapp, C. H., & Carter, G. C. (1976). The Generalized Correlation Method for Estimation of Time Delay. *IEEE Transactions on Acoustics, Speech and Signal*

Processing.

- Leslie, J. *Noise Levels And Stealth Features Of DJI Mini 4 Pro.* (s/f). Sky-Kam. Recuperado el 13 de julio de 2026. <https://skykam.co.uk/how-loud-is-dji-mini-4-pro/>
- Li, X., Deng, Z. D., Rauchenstein, L. T., & Carlson, T. J. (2016). Contributed Review: Source-localization algorithms and applications using time of arrival and time difference of arrival measurements. *Review of Scientific Instruments*, 87(4), 041502. doi: 10.1063/1.4947001
- Ramamonjy, A., Bavu, E., Garcia, A., & Hengy, S. (2018). *Source localization and identification with a compact array of digital MEMS microphones.*
- Riabko, A. V., Vakaliuk, T. A., Zaika, O. V., Kukharchuk, R. P., & Kontsedailo, V. V. (2024). *Edge computing applications: Using a linear MEMS microphone array for UAV position detection through sound source localization.*
- Schmidt, R. (1986). Multiple Emitter Location and Signal Parameter Estimation. *IEEE Transactions on Antennas and Propagation*, 34(3), 276–280. doi: 10.1109/TAP.1986.1143830
- Van Trees, H. L. (2001). *Detection, estimation, and modulation theory.* Wiley.
- Van Trees, H. L. (2002). *Optimum Array Processing: Part IV of Detection, Estimation, and Modulation Theory* (1.^a ed.). John Wiley & Sons.
- Xu, E., Ding, Z., & Dasgupta, S. (2011). Source Localization in Wireless Sensor Networks From Signal Time-of-Arrival Measurements. *IEEE Transactions on Signal Processing*, 59(6), 2887–2897. doi: 10.1109/TSP.2011.2116012
- Zhang, X., Huang, J., Song, E., Liu, H., Li, B., & Yuan, X. (2014). Design of Small MEMS Microphone Array Systems for Direction Finding of Outdoors Moving Vehicles. *Sensors*, 14(3), 4384–4398. doi: 10.3390/s140304384
- Zhu, J., Cheng, R., Li, J., Tian, Y., & Zhang, Y. (2021). Sound source location for low-altitude aircraft based on sub-band extraction. *MATEC Web of Conferences*, 336, 01004. doi: 10.1051/mateconf/202133601004